

UNIVERSITATEA „LUCIAN BLAGA” DIN SIBIU
FACULTATEA DE ȘTIINȚE

Specializarea: Informatică

LUCRARE DE LICENȚĂ

Coordonator științific
Conf. univ. dr. Florin STOICA

Absolvent
Sorina CIOBANU



Sibiu
2026

UNIVERSITATEA „LUCIAN BLAGA” DIN SIBIU
FACULTATEA DE ȘTIINȚE

Specializarea: Informatică

**SISTEM INFORMATIC PENTRU
MANAGEMENTUL OPERAȚIUNILOR ÎNTR-
UN CENTRU DE DAUNE AUTO**

Coordonator științific
Conf. univ. dr. Florin STOICA

Absolvent
Sorina CIOBANU



Sibiu
2026

Declarație pentru conformitate asupra originalității operei științifice



UNIVERSITATEA
LUCIAN BLAGA
— DIN SIBIU —

Ministerul Educației și Cercetării
Universitatea "Lucian Blaga" din Sibiu

Facultatea de Științe

Conf. univ. dr. Florin Stoica

Flu

Anexa 2

DECLARAȚIA PENTRU CONFORMITATE ASUPRA ORIGINALITĂȚII OPEREI ȘTIINȚIFICE

Subsemnatul /Subsemnata CIORANU SORINA,
domiciliat/ă în localitatea ATEL, adresa STR. PRINCIPALĂ, NR. 32, având actul de
identitate seria SR nr. 058412, codul numeric personal 6050410323946, înscris/ă pentru
examenul de licență / proiect de diplomă / disertație / susținere teză de doctorat, cu tema:
SISTEM INFORMATIC PENTRU MANAGEMENTUL OPERAȚIUNILOR
ÎNTR-UN CENTRU DE MAUNE AUTO

declar următoarele:

- Opera științifică nu aparține altei persoane, instituții, entități cu care mă aflu în relații de muncă sau altă natură.
- Am înțeles și respect prevederile Codului de Etică și Deontologie Universitară al ULBS și ale legislației în vigoare privind integritatea academică, în special cele referitoare la plagiat și fraudă academică.
- Îmi asum responsabilitatea deplină pentru originalitatea lucrării și pentru orice abatere de la normele de integritate academică.

De asemenea, declar pe proprie răspundere următoarele, în legătură cu utilizarea instrumentelor de inteligență artificială (AI) în elaborarea prezentei lucrări:

[] Lucrarea a fost elaborată exclusiv prin contribuție intelectuală proprie, fără utilizarea de instrumente de inteligență artificială (AI). (Se bifează dacă este cazul)

☒ Lucrarea a fost elaborată cu sprijinul instrumentelor de inteligență artificială (AI), caz în care declar și detaliez, conform celor de mai jos, modul în care au fost utilizate aceste instrumente și asigur că ele au servit doar ca unelte de asistență, iar conținutul și ideile fundamentale îmi aparțin integral: (Se bifează dacă este cazul și se completează secțiunea de mai jos)

- Tipul/Tipurile de instrument/e AI utilizat/e (ex: ChatGPT, Google Bard, Grammarly AI, alte unelte specifice, etc.):

GEMINI
.....
.....



UNIVERSITATEA
LUCIAN BLAGA
— DIN SIBIU —

Ministerul Educației și Cercetării
Universitatea "Lucian Blaga" din Sibiu

Facultatea de Științe

• Scopul utilizării instrumentelor AI (ex: generare de idei inițiale, structurare generală, reformulare de propoziții/paragrafe, verificare gramaticală și stilistică, generare de cod/date, analiză de date, rezumate, traduceri, etc.):

• VERIFICAREA GRAMATICĂ, CORECTAREA
STILISTICĂ, REFORMULARE DE PROPOZIȚII

• Detalierea modului în care conținutul generat sau asistat de AI a fost verificat, adaptat și integrat, asigurând originalitatea, coerența intelectuală și proprietatea asupra ideilor și argumentației din lucrare:

• TOATE SUGESTIILE AU FOST ANALIZATE ȘI
INTEGRATE MANUAL ÎN TEXT

Confirm că am înțeles și respect obligația de a asigura originalitatea integrală a conținutului, asumându-mi deplina responsabilitate pentru întreaga lucrare prezentată.

Data: 24.06.2026

Numele și prenumele: CIOBANU SORINA

Semnătura:

Cuprins

Introducere	1
Obiectivele lucrării.....	1
Structura lucrării	2
Capitolul 1. Managementul operațiunilor într-un centru de daune auto	3
1.1. Centrul de daune auto, rol și funcții în cadrul industriei asigurărilor auto	3
1.2. Procese operaționale specifice unui centru de daune auto	4
1.2.1. Înregistrarea și gestionarea dosarelor de daună	4
1.2.2. Evaluarea tehnică a daunelor	4
1.2.3. Planificarea și monitorizarea reparațiilor	4
1.2.4. Comunicarea cu clienții și companiile de asigurări	5
1.3. Necesitatea digitalizării proceselor operaționale	5
1.4. Rolul sistemelor informatice în optimizarea activităților operaționale.....	6
1.5. Avantajele utilizării aplicațiilor informatice în managementul daunelor auto	6
Capitolul 2. Analiza pieței și studiu comparativ al aplicațiilor existente	8
2.1. Necesități și cerințe ale sistemelor informatice utilizate în domeniul gestionării daunelor	8
2.2. Prezentarea unor aplicații software existente.....	8
2.3. Analiza comparativă a soluțiilor existente.....	9
2.4. Identificarea limitărilor sistemelor actuale	10
2.5. Motivația dezvoltării unei noi aplicații informatice.....	11
Capitolul 3. Tehnologii utilizate în dezvoltarea aplicației	13
3.1. Limbajul de programare Java și utilizarea acestuia în dezvoltarea aplicațiilor enterprise	13
3.2. Framework-uri și tehnologii pentru dezvoltarea backend-ului	13
3.2.1. Arhitectura aplicațiilor web bazate pe Java	14
3.2.2. Implementarea serviciilor REST.....	14
3.3. Biblioteca React.js pentru dezvoltarea interfețelor utilizator	14
3.4. Sistemul de gestiune a bazelor de date MySQL	15

3.5. Comunicarea între componentele aplicației prin API REST	16
3.6. Avantajele utilizării arhitecturii Java – React.js – MySQL	17
Capitolul 4. Analiza și proiectarea sistemului informatic	18
4.1. Identificarea cerințelor funcționale ale sistemului	18
4.2. Identificarea cerințelor nefuncționale	18
4.3. Arhitectura generală a aplicației	19
4.4. Modelarea sistemului informatic	20
4.4.1. Diagrama cazurilor de utilizare	20
4.4.2. Diagrama componentelor sistemului	21
4.4.3. Diagrama fluxului de date	22
4.5. Proiectarea bazei de date	23
4.5.1 Modelul conceptual al bazei de date	24
4.5.2 Modelul logic al bazei de date	24
4.5.3 Structura tabelelor și relațiile dintre acestea	24
4.6. Definirea rolurilor utilizatorilor în sistem	26
4.6.1. Administrator	26
4.6.2. Operator centru de daune	26
4.6.3. Inspector de daună	26
4.6.4. Personal service auto	26
4.6.5 Client	27
4.7. Proiectarea interfețelor utilizator	27
Capitolul 5. Implementarea aplicației	29
5.1. Configurarea mediului de dezvoltare	29
5.2. Implementarea componentei backend în Java	29
5.2.1. Definirea modelelor de date	29
5.2.2. Implementarea logicii de business	31
5.2.3. Implementarea serviciilor REST	32
5.3. Implementarea bazei de date MySQL	33

5.4. Implementarea interfeței utilizator folosind React.js	34
5.4.1. Modulul de autentificare și autorizare	34
5.4.2. Modulul de gestionare a clienților	34
5.4.3. Modulul de gestionare a vehiculelor	35
5.4.4. Modulul de administrare a dosarelor de daună	35
5.4.5. Modulul de programare și monitorizare a reparațiilor	36
5.4.6. Modulul de rapoarte și statistici	37
5.5. Integrarea componentelor aplicației	38
Capitolul 6. Utilizarea sistemului informatic	39
6.1. Autentificarea în aplicație	39
6.2. Gestionarea datelor despre clienți și vehicule	39
6.3. Administrarea dosarelor de daună	41
6.4. Monitorizarea stadiului lucrărilor	42
6.5. Generarea rapoartelor	43
6.6. Exemple de scenarii de utilizare	44
Capitolul 7. Evaluarea și analiza rezultatelor și beneficiilor aplicației	46
7.1. Beneficiile utilizării sistemului informatic	46
7.2. Impactul asupra eficienței operaționale	46
7.3. Limitările aplicației dezvoltate	48
7.4. Direcții de dezvoltare ulterioară	48
Concluzii	50
Bibliografie	51

Introducere

Fluxurile de lucru din majoritatea industriilor tradiționale au suferit transformări profunde sub impactul tranziției digitale, un domeniu parțial imun la această tendință rămânând însă cel al asigurărilor auto. Gestiunea administrativă din numeroase centre de evaluare stă încă sub semnul completării manuale a formularelor și al dependenței de documente fizice. Procesarea dosarelor de daună devine astfel anevoioasă, suportul de hârtie amplificând riscul erorilor umane și generând blocaje operaționale greu de tolerat în dinamismul economic actual.

Dezvoltarea unei platforme informatice integrate a apărut ca o necesitate practică pentru a rezolva un paradox tehnologic flagrant: deși autovehiculele moderne încorporează sisteme inteligente de ultimă generație, gestionarea daunelor continuă să fie marcată de un caracter anacronic. Comunicarea fragmentată între service-uri, inspectori și asigurători, alături de utilizarea hârtiilor de deteriorare desenate pe hârtie, amână nejustificat plata despăgubirilor.

Din perspectivă tehnică, edificarea unei aplicații web complexe a oferit oportunitatea aplicării riguroase a bunelor practici din ingineria software contemporană. S-a optat pentru o arhitectură full-stack decuplată, extinsă de la nivelul bazei de date relaționale până la interfața grafică. Securitatea datelor și respectarea regulilor de business sunt garantate printr-un mecanism strict de autentificare și autorizare structurat pe cinci roluri distincte, incluzând administratorul, operatorul, inspectorul de daună și personalul de service.

Obiectivele lucrării

Scopul principal al lucrării constă în proiectarea și implementarea unei platforme informatice destinate managementului integrat al dosarelor de daună auto. Realizarea acestui system a impus îndeplinirea următoarelor obiective specifice:

- Dezvoltarea unei infrastructuri backend securizate: Această componentă asigură gestiunea relațiilor logice complexe dintre entitățile bazei de date și garantează protecția informațiilor stocate.
- Proiectarea unei interfețe grafice responsive: Subsistemul de frontend oferă o experiență utilizator intuitivă, fiind adaptat fluxurilor de lucru specifice fiecărui rol definit în aplicație.
- Implementarea unui modul de analiză și raportare: Această funcționalitate permite generarea în timp real a statisticilor manageriale necesare monitorizării activității.
- Asigurarea persistenței și integrității datelor: Obiectivul transversal al proiectului a vizat optimizarea accesului concurent, prevenind conflictele de scriere în condițiile utilizării simultane a platformei de către mai mulți operatori.

Structura lucrării

Pentru a prezenta coerent toate aspectele menționate, lucrarea a fost structurată în șapte capitole distincte, ordonate într-o succesiune logică și tehnică.

Primul capitol stabilește cadrul operațional al lucrării, analizând rolul centrelor de daune în piața asigurărilor, particularitățile gestionării dosarelor și motivele pentru care digitalizarea acestui proces devine o necesitate practică în contextul actual.

Capitolul al doilea realizează un studiu de piață și o analiză comparativă a soluțiilor software existente în industrie, evidențiind limitele acestora și justificând astfel necesitatea dezvoltării unei noi platforme digitale adaptate.

În al treilea capitol sunt prezentate tehnologiile utilizate în procesul de dezvoltare, cu o descriere a limbajului Java pentru aplicații enterprise, a framework-urilor backend, a bibliotecii React.js și a bazei de date MySQL, precum și a avantajelor comunicării prin API REST.

Capitolul al patrulea este dedicat analizei și proiectării sistemului, cuprinzând cerințele funcționale, diagramele de structură, fluxurile de date, modelul logic al bazei de date și definirea rolurilor utilizatorilor.

Al cincilea capitol descrie etapele de implementare efectivă, cu detalii privind scrierea codului în Spring Boot și React.js, configurarea modulelor de securitate, structurarea bazei de date și testarea generală a aplicației.

Capitolul al șaselea funcționează ca un ghid practic de utilizare a sistemului informatic, exemplificând scenarii concrete de lucru și prezentând interfețele aplicației pentru fiecare tip de utilizator autentificat.

Ultimul capitol conține evaluarea rezultatelor obținute, analiza impactului aplicației asupra eficienței operaționale, limitele tehnice actuale ale prototipului și direcțiile avute în vedere pentru dezvoltările viitoare ale platformei.

Capitolul 1. Managementul operațiunilor într-un centru de daune auto

1.1. Centrul de daune auto, rol și funcții în cadrul industriei asigurărilor auto

Aplicația propusă pentru centrul de evaluare a daunelor auto urmărește, în esență, două obiective care se completează: centralizarea informațiilor relevante pentru fiecare dosar și fluidizarea comunicării dintre client și compania de asigurări.

În structura actuală a pieței de asigurări rutiere, centrul de daune funcționează ca intermediar între trei categorii de actori: societatea de asigurări, unitatea de service auto și clientul păgubit. Rolul acestuia s-a schimbat considerabil față de perioada în care activitatea se rezuma la preluarea și transmiterea documentelor pe suport fizic. Astăzi, centrul are responsabilitatea de a valida tehnic și economic fiecare eveniment rutier înregistrat, ceea ce îi conferă o poziție semnificativ mai complexă în lanțul procesului de despăgubire.

Funcția principală a centrului constă în gestionarea procesului de constatare și evaluare a avariilor, un flux de activități care debutează imediat după producerea accidentului. O etapă esențială este verificarea concordanței dintre declarațiile conducătorilor auto și deteriorările efectiv constatate pe vehicul, verificare care presupune o analiză comparativă strictă a informațiilor disponibile. Din perspectiva sistemului informatic, această cerință implică nu doar stocarea și prelucrarea datelor, ci și mecanisme de control prin care despăgubirile solicitate să fie raportate la normele tehnice ale producătorilor și la istoricul dosarelor existente. Verificările încrucișate reprezintă principalul instrument de limitare a riscului de fraudă, iar eficiența lor depinde direct de calitatea și completitudinea datelor introduse în sistem. Prin urmare, arhitectura aplicației trebuie să faciliteze colectarea structurată a informațiilor din toate etapele procesului.

Medierea comunicării între părțile implicate este de asemenea o funcție importantă. Centrul procesează date din surse diverse: procese-verbale de poliție, formulare de constatare amiabilă, polițe RCA sau CASCO și fișe tehnice de la producătorul vehiculului. Pe baza acestora se stabilește valabilitatea dosarului și se emite nota de constatare, document fără de care nu pot fi autorizate lucrările de reparație și nici decontate facturile emise de service.

Din perspectivă economică, controlul costurilor reprezintă un alt obiectiv central al centrului de daune. Utilizarea cataloagelor de piese actualizate și aplicarea unor metode standardizate de calcul reduc riscul supraevaluării devizelor de reparație, o problemă frecvent semnalată în literatura de specialitate privind gestionarea daunelor auto [1]. Reducerea timpului de imobilizare a vehiculului în service contribuie, la rândul ei, la satisfacția clientului și, implicit, la imaginea companiei de asigurări în fața consumatorilor. Într-un context în care așteptările legate de rapiditate și transparență în procesul de despăgubire sunt tot mai ridicate, capacitatea centrului de a răspunde acestor cerințe devine un factor diferențiator relevant.

1.2. Procese operaționale specifice unui centru de daune auto

1.2.1. Înregistrarea și gestionarea dosarelor de daună

Inițierea operațională a unui nou dosar constituie primul punct de interacțiune directă între un potențial client și sistemul de management implementat. Atunci când un vehicul avariat ajunge în cadrul centrului de evaluare, utilizatorul are sarcina de a completa un formular electronic structurat, introducând datele cu caracter personal și detaliile aferente accidentului, existând de asemenea opțiunea ca dosarul de daună să fie deschis direct de către păgubit prin intermediul contului său de utilizator. Colectarea documentelor de identitate, a polițelor de asigurare, precum și a declarațiilor pe proprie răspundere se realizează în format electronic. Aplicația web generează o identitate digitală unică pentru respectivul incident în momentul în care aceste date sunt salvate în tabelele bazei de date. Toate fluxurile operaționale ulterioare se vor construi având ca ancoră acest identificator unic. Administrarea corectă a informațiilor colectate deține o importanță capitală, deoarece omiterea sau introducerea eronată a unui singur element în această etapă incipientă poate conduce ulterior la blocarea întregului proces de despăgubire în cadrul sistemului informatic al asigurătorului.

1.2.2. Evaluarea tehnică a daunelor

După ce informațiile inițiale au fost salvate și transmise cu succes către biroul de daune, sistemul trece în etapa de verificare a conformității pentru validarea datelor introduse de către client. Inspekția fizică propriu-zisă a autovehiculului avariat este inițiată doar dacă toate documentele încărcate în aplicație sunt conforme. Inspectorul de daune examinează automobilul cu o atenție sporită, urmărind identificarea urmelor vizibile de impact, a componentelor îndoite sau a mecanismelor interne distruse în urma accidentului rutier. Această evaluare depășește stadiul unei simple examinări sumare, fundamentându-se pe criterii tehnice riguroase care sunt stabilite de producătorii auto. Se realizează o demarcație extrem de clară între elementele care pot fi reparate în condiții de siguranță deplină și piesele care impun o înlocuire integrală cu repere noi, de fabrică.

Precizia datelor colectate reprezintă factorul decisiv în acest punct al fluxului de lucru. Orice eroare minoră sau omisiune apărută în faza de constatare va atrage după sine cheltuieli suplimentare în faza de reparație în service. Apariția unor astfel de cheltuieli neprevăzute constituie un factor de dezechilibru pentru stabilitatea financiară a afacerii. Din acest motiv, introducerea corectă a reperelor în modulele aplicației garantează obținerea unui deviz final calculat corect, eliminând riscurile asociate factorului uman și asigurând un proces sigur și transparent.

1.2.3. Planificarea și monitorizarea reparațiilor

Înregistrarea completă a tuturor avariilor în baza de date declanșează solicitarea sistemului de a genera un plan de acțiune explicit. Platforma coordonează în mod direct programul mecanicilor din atelierul de service și analizează automat disponibilitatea pieselor de schimb în

depozite, prevenind astfel apariția unor blocaje administrative printr-o monitorizare constantă. Este fundamental ca operatorul să cunoască în fiecare moment dacă vehiculul se află în etapa de vopsitorie sau dacă echipa tehnică efectuează montajul noilor componente. Minimizarea timpilor morți determină o returnare mult mai rapidă a automobilului către proprietarul păgubit, acest aspect reprezentând un beneficiu major atât pentru client, cât și pentru centrul de daune auto.

1.2.4. Comunicarea cu clienții și companiile de asigurări

Menținerea unui flux comunicațional neîntrerupt între clienți și societățile de asigurare reprezintă una dintre premisele fundamentale ale derulării procesului de despăgubire. Centrul de daune își asumă rolul de mediator între aceste două entități, având responsabilitatea de a traduce informațiile pur tehnice în mesaje ușor de înțeles pentru proprietarul mașinii.

Proprietarul are nevoie să primească actualizări în timp real cu privire la stadiul reparației, fără a fi constrâns să interpreteze terminologia specifică ingineriei auto sau prevederile legislației în vigoare. Absența unei asemenea transparențe favorizează apariția neînțelegerilor și determină solicitări repetate de informații, elemente care îngreunează activitatea de zi cu zi a operatorilor din centru.

Relația stabilită cu societatea de asigurări se supune unor norme complet diferite, ce reclamă un nivel înalt de rigoare formală. Transmiterea rapidă a devizelor detaliate, a rapoartelor tehnice de constatare și a întregii documentații justificative este strict obligatorie în vederea aprobării fondurilor destinate plății. Orice neconcordanță constatată între avariile fizice identificate și cerințele impuse de asigurător poate determina respingerea dosarului, generând consecințe directe negative pentru toate părțile implicate. Prin urmare, automatizarea sistemului de notificări și centralizarea documentelor în aplicație contribuie direct la reducerea erorilor de transmitere, scurtând semnificativ ciclul necesar pentru aprobarea despăgubirilor.

1.3. Necesitatea digitalizării proceselor operaționale

Menținerea fluxurilor operaționale exclusiv în zona tradițională a documentelor tipărite reprezintă o vulnerabilitate structurală majoră pentru centrele moderne care gestionează daunele auto. Blocajele se instalează rapid în aceste condiții. Când circulația datelor depinde în întregime de suportul analogic, volumele masive de hârtie acumulate pe birouri provoacă întârzieri administrative severe, ceea ce obligă clientul să aștepte perioade îndelungate pentru decizii care întârzie să apară. Această dependență pronunțată de suportul fizic limitează drastic viteza de reacție a întregii companii. Informațiile se pot pierde cu ușurință. Transportarea fizică a dosarelor între departamente sau scanarea repetată urmată de trimiterea acestora prin mesaje electronice interminabile și nestructurate sporesc exponențial riscul apariției erorilor umane, afectând grav coerența datelor salvate. Tranziția către digital nu constituie o simplă preferință estetică, ci o cerință fundamentală pentru supraviețuirea într-un mediu economic extrem de competitiv. Acest proces redefinesc total modul în care utilizatorul interacționează cu societatea de asigurări. Dosarele uitate prin sertare sunt complet eliminate, locul acestora fiind preluat de date structurate care pot fi accesate printr-un singur click.

Digitalizarea sporește semnificativ controlul global pe care managementul companiei îl exercită asupra evidenței tuturor cazurilor înregistrate în sistem. Nivelul de transparență crește imediat. Organizarea informațiilor în cadrul unei arhitecturi relaționale stabile oferă posibilitatea identificării instantanee a punctelor critice în care un anumit dosar este blocat din cauza lipsei unui document justificativ sau a unei aprobări specifice. Echipa managerială obține astfel capacitatea de a adopta decizii corecte bazate pe indicatori cifrici și rapoarte tehnice reale, renunțând la simple supoziții ori estimări aproximative. Practic, transformarea digitală reușește să înlocuiască haosul administrativ specific modelului tradițional cu un flux de lucru predictibil, eficient și extrem de ușor de auditat la nivel de business, oferind avantaje incontestabile atât pentru angajați, cât și pentru clientul final.

1.4. Rolul sistemelor informatice în optimizarea activităților operaționale

Integrarea sistemelor informatice în managementul operațional diminuează considerabil timpul alocat sarcinilor administrative, în special prin eliminarea completării manuale a documentelor pe suport de hârtie. Interconectarea tuturor actorilor implicați în cadrul unei platforme comune, prevăzute cu panouri de control distincte configurate în funcție de rolul fiecărui utilizator, aduce o vizibilitate net superioară asupra fluxului de lucru și facilitează urmărirea fiecărui dosar individual. Păgubitul nu mai este constrâns să efectueze deplasări fizice sau să aștepte la ghișee, dat fiind că întreaga interacțiune poate fi administrată în mediul online. Generarea automată a rapoartelor și completarea formularelor electronice fără a fi necesară reintroducerea repetată a acelorași date diminuează efortul depus de operatori și previne apariția erorilor de transcriere.

Informațiile sunt stocate corect încă de la prima interacțiune cu sistemul, fapt care garantează o organizare mult mai coerentă a dosarelor active și un timp de răspuns considerabil mai redus în comparație cu modelele tradiționale.

1.5. Avantajele utilizării aplicațiilor informatice în managementul daunelor auto

Modelele tradiționale folosite pentru gestionarea daunelor auto, bazate pe documente fizice, interacțiuni telefonice și procese de lucru fragmentate, determină costuri operaționale substanțiale și întârzieri greu de controlat.

Adoptarea unor soluții software integrate abordează aceste deficiențe din perspective multiple. Centralizarea întregului istoric al unui dosar într-o singură bază de date securizată elimină transmiterea necontrolată a informațiilor între păgubit, broker, inspector și unitatea de service, diminuând substanțial riscurile de apariție a erorilor de transcriere. Notificarea electronică a unui nou incident declanșează automat alocarea dosarului către un inspector disponibil, în timp ce alertele automate mențin toate părțile informate fără a necesita vreo intervenție manuală adițională. Un efect direct măsurabil este reprezentat de reducerea timpului mediu necesar reparației, indicator cunoscut în industrie sub denumirea de cycle time [1]. Interconectarea digitală realizată între inspectori și atelierele de reparații permite transmiterea, dar și aprobarea devizelor

în tempo real, eliminând zilele de așteptare care caracterizează corespondența clasică purtată prin poștă electronică sau servicii de curierat. Clientul este permanent informat cu privire la stadiul dosarului său, de la faza de analiză până la cea de aprobare și execuție finală, proces ce reduce totodată volumul solicitărilor adresate centrelor de suport telefonic.

Acuratețea ridicată a datelor colectate aduce o contribuție esențială la prevenirea tentativelor de fraudă [2]. Stocarea structurată a imaginilor prevăzute cu amprentă de geolocalizare, a documentelor scanate și a rapoartelor tehnice detaliate face posibile verificări încrucișate automate înainte de efectuarea oricărei plăți.

Din perspectivă managerială, volumul extins de date acumulate oferit de sistem susține analize statistice deosebit de utile pentru optimizarea întregii activități. Managerii au posibilitatea de a extrage rapoarte complexe referitoare la eficiența unităților de reparații, costurile medii calculate pe categorii de vehicule și viteza de răspuns a personalului din teren, toate aceste informații fundamentând decizii operaționale mult mai bine documentate.

Capitolul 2. Analiza pieței și studiu comparativ al aplicațiilor existente

2.1. Necesități și cerințe ale sistemelor informatice utilizate în domeniul gestionării daunelor

Proiectarea unei platforme informatice dedicate managementului daunelor auto presupune o înțelegere detaliată a tuturor interacțiunilor și a fluxurilor de lucru stabilite în mod curent între societățile de asigurări, persoanele pagubite și unitățile de reparații. Natura complexă a acestui sector impune adoptarea unor soluții software flexibile, capabile să gestioneze volume considerabile de date eterogene, de la documente juridice și fișe tehnice până la imagini de mari dimensiuni încărcate direct din browser, fără a fi compromisă securitatea sau trasabilitatea informațiilor. Analiza cerințelor actuale ale pieței evidențiază faptul că eficiența unui astfel de sistem depinde în mod direct de trei aspecte principale, anume automatizarea fluxului de lucru, interoperabilitatea componentelor și accesibilitatea datelor.

O necesitate operațională stringentă o constituie reducerea timpului mediu solicitat pentru soluționarea unui dosar, indicator cunoscut în industrie sub denumirea de cycle time. Ecosistemele tradiționale se confruntă frecvent cu o fragmentare pronunțată a informațiilor, deoarece datele rămân adesea dispersate în mesaje e-mail, apeluri telefonice repetate sau dosare fizice greu de urmărit de la distanță. Din această cauză, o cerință de bază constă în realizarea unei arhitecturi web capabile să reunească toți actorii implicați într-un flux digital continuu, controlat și complet verificabil.

Aplicațiile moderne trebuie să asigure o comunicare în regim asincron, prin intermediul unor API-uri de tip REST [6], cu baze de date externe, sisteme financiare destinate facturării și cataloage electronice de prețuri pentru piesele auto, garantând consistența deplină a informațiilor fără a suprasolicita resursele serverului de backend. Totodată, sistemul are obligația de a oferi interfețe web adaptate fiecărui rol în parte și de a proteja datele cu caracter personal prin module riguroase de autentificare și autorizare corespunzătoare.

2.2. Prezentarea unor aplicații software existente

Pentru a înțelege standardele tehnologice actuale și bunele practici din industrie, este utilă o analiză a celor mai reprezentative soluții software folosite la nivel global în managementul daunelor auto, fiecare dintre acestea abordând diferit organizarea și controlul fluxurilor de date.

Audatex constituie în prezent un reper recunoscut în domeniul evaluărilor auto [3], succesul său bazându-se pe o bază de date extinsă care include schițe tehnice detaliate și coduri oficiale de producător pentru marea majoritate a vehiculelor aflate în circulație. Platforma este orientată spre maximizarea preciziei în calculul pieselor și al manoperei, motiv pentru care rămâne instrumentul principal utilizat de inspecitori și de atelierele de reparații pentru generarea devizelor oficiale solicitate de companiile de asigurări. Cu toate acestea, interfața prezintă o complexitate ridicată și o anumită rigiditate structurală, fiind o soluție concepută aproape exclusiv pentru

utilizatorii experți și mai puțin accesibilă clientului obișnuit care are nevoie de un parcurs simplu și transparent.

Guidewire (ClaimCenter) este o altă soluție de anvergură, o platformă de nivel enterprise proiectată pentru marile corporații din domeniul asigurărilor [4]. Sistemul preia și gestionează întregul ciclu de viață al unei daune, de la validarea inițială a poliței până la finalizarea plăților, cu accent pe automatizarea procesului decizional, detectarea fraudelor și managementul financiar complex. Fiind o soluție de mari dimensiuni, implementarea implică investiții considerabile în licențiere și infrastructură dedicată, ceea ce face dificilă adaptarea sa rapidă la fluxuri de lucru personalizate sau utilizarea directă de către unități de service mici și independente.

Snapsheet abordează piața dintr-o perspectivă diferită, fiind o platformă modernă construită în contextul tehnologiilor cloud [5], recunoscută pentru introducerea conceptului de evaluare virtuală. Strategia sa se bazează pe o abordare de tip mobile-first, oferind utilizatorilor posibilitatea de a fotografia avariile și de a deschide un dosar nou direct de pe smartphone. Aplicația folosește fluxuri rapide de procesare în cloud pentru a accelera aprobările financiare și, prin modul de interacțiune asincronă cu utilizatorul, se apropie cel mai mult de filozofia aplicațiilor web contemporane de tip Single Page Application.

2.3. Analiza comparativă a soluțiilor existente

Analiza comparativă a platformelor Audatex, Guidewire și Snapsheet evidențiază viziuni arhitecturale diferite, fiecare soluție fiind concepută pentru a răspunde unei anumite nișe din piața asigurărilor auto. Audatex excelează în calculele tehnice și identificarea pieselor de schimb, însă nu oferă un flux complet pentru gestionarea interacțiunii directe cu beneficiarul final. Guidewire asigură o administrare financiară și operațională solidă pentru marile corporații, dar prezintă limitări la capitolul flexibilitate, din cauza unei interfețe complexe și a unor costuri de achiziție și mentenanță ridicate. Snapsheet aduce mobilitate și o orientare clară spre utilizator prin adoptarea tehnologiilor cloud, chiar dacă nu include precizia tehnică a unui catalog de piese integrat nativ.

În acest context, arhitectura propusă în lucrarea de față, dezvoltată pe o stivă tehnologică formată din Java (Spring Boot), React.js și MySQL, se justifică printr-o poziționare hibridă, flexibilă și avantajoasă din punct de vedere economic. Spre deosebire de sistemele monolitice sau de aplicațiile comerciale cu licențe enterprise, utilizarea React.js asigură o interfață modernă și interactivă, similară cu experiența oferită de aplicațiile cloud actuale, în timp ce backend-ul în Java garantează un nivel adecvat de securitate și robustețe în procesarea datelor, toate informațiile fiind organizate într-o bază de date relațională administrată prin MySQL.

Tabelul următor centralizează structura tehnică, avantajele și limitările aplicațiilor analizate în raport cu soluția dezvoltată în acest proiect:

Criteriu	Audatex	Guidewire	Snapshot	Soluția propusă (Java-React)
Arhitectura software	Monolit cu integrare Cloud	Enterprise (SaaS/On-Premise)	Cloud-Native bazat pe Mobile API	Arhitectură decuplată pe trei straturi (React + Spring Boot)
Publicul țintă	Inspectori auto și unități service	Companii mari de asigurări	Asiguratori și clienți finali	Clienți, operatori, inspecții și personal service
Principalul avantaj	Precizie în calculul pieselor și manoperei	Gestiune financiară masivă și audit complet	Deschidere rapidă a dosarului direct de pe telefon	Interfață fluidă (SPA), flexibilitate și cost zero de licență
Principala limitare	Interfață rigidă, fără flux de dialog cu clientul	Costuri mari și implementare complexă	Dependență de ecosistemul propriu găzduit în cloud	Necesită integrare manuală cu nomenclatoare externe de piese
Model de cost	Plată per utilizare sau licență tehnică	Licență enterprise cu costuri ridicate	Abonament periodic de tip SaaS	Open-source cu costuri minime de găzduire
Grad de personalizare	Scăzut, sistem închis	Mediu, necesită consultanți dedicați	Mediu, configurabil prin API	Ridicat, cod sursă complet adaptabil

Tabelul 2.1. Analiză comparativă între soluțiile comerciale existente și soluția propusă.

Acest tabel evidențiază faptul că proiectul propus reușește să combine avantajele de utilizare ale unei aplicații moderne orientate spre client cu securitatea și structura necesare unui mediu de lucru cu roluri multiple. Eliminarea costurilor de licențiere transformă această stivă software într-o alternativă viabilă pentru centrele de daune care au nevoie de o soluție adaptată propriilor fluxuri operaționale.

2.4. Identificarea limitărilor sistemelor actuale

Studiul platformelor consacrate în managementul daunelor auto, precum Audatex, Guidewire sau Snapshot, relevă o serie de bariere tehnice și operaționale care afectează eficiența proceselor. Aceste soluții performează bine pe nișele pentru care au fost proiectate, dar nu reușesc să acopere în mod unitar nevoile tuturor actorilor implicați, categorie care reunește societățile de asigurări, inspectorii de teren, atelierele de reparații și clienții finali.

Deficiențele identificate pot fi grupate în câteva direcții distincte, care pun în lumină necesitatea unor abordări alternative.

Rigiditatea arhitecturală reprezintă una dintre problemele comune întâlnite pe aceste platforme. Multe aplicații tradiționale din acest sector sunt construite pe structuri de tip monolit sau pe tehnologii cu vechime, fiind extrem de greu de modificat fără riscuri majore. Orice ajustare a unui flux de lucru existent sau introducerea unei funcționalități noi devine un proces lent și costisitor, capabil să destabilizeze întregul sistem, ceea ce face ca adaptarea la cerințele noi ale pieței să fie anevoioasă în aceste condiții.

O altă barieră provine din fragmentarea publicului țintă, având în vedere că Audatex este folosit aproape exclusiv de unitățile service și de inspectori pentru calculul tehnic, în timp ce Guidewire deservește operațiunile financiare interne ale asigurătorilor. Această segmentare conduce la omiterea clientului final din cadrul sistemelor informatice, acesta fiind obligat să solicite informații prin canale externe precum telefonul sau poșta electronică, fapt care introduce întârzieri și reduce transparența generală a procesului.

Costurile ridicate reprezintă o limitare concretă cu un impact major asupra accesibilității. Soluțiile enterprise implică bugete de achiziție substanțiale, procese complexe de adaptare la nevoile specifice și servicii de mentenanță continuă, aceste cheltuieli fiind greu de amortizat pentru un service independent sau pentru o companie de asigurări de dimensiuni medii. În practică, accesul la aceste platforme rămâne un avantaj exclusiv al marilor corporații.

Nu în ultimul rând, experiența utilizatorului lasă de dorit în multe situații. Interfețele multor aplicații din domeniu sunt dense, caracterizate de o curbă de învățare abruptă, solicitând perioade lungi de instruire a personalului. Absența unor interfețe web moderne, construite dintr-o singură pagină fluidă ce elimină reîncărcarea paginilor, diminuează rata de adoptare și îngreunează executarea sarcinilor zilnice.

2.5. Motivația dezvoltării unei noi aplicații informatice

Limitările structurale și tehnice identificate la nivelul soluțiilor existente pe piață au constituit argumentul principal care a stat la baza proiectării și dezvoltării aplicației informatice prezentate în această lucrare. Nevoia unui instrument digital integrat, flexibil și accesibil din punct de vedere financiar a oferit contextul potrivit pentru implementarea unei soluții moderne. Unul dintre obiectivele importante ale acestui demers este eliminarea barierelor financiare asociate licențierii, prin utilizarea exclusivă a tehnologiilor cu sursă deschisă. Astfel, companiile de dimensiuni medii și rețelele independente de service-uri auto pot adopta o soluție digitală fără a fi supuse unor presiuni financiare semnificative, ceea ce facilitează modernizarea serviciilor proprii.

Spre deosebire de sistemele fragmentate care izolează utilizatorii în aplicații separate, noul sistem a fost conceput pentru a reuni toți actorii implicați într-un singur flux operațional coerent. Această unificare este realizată prin implementarea unui model de control al accesului bazat pe roluri, care permite administratorului, clientului, operatorului, inspectorului și personalului din service să interacționeze în condiții de siguranță în cadrul aceleiași platforme. Fiecare acțiune este înregistrată în sistem, asigurând trasabilitatea documentelor și coordonarea între departamente, fără a mai fi necesară utilizarea unor canale de comunicare externe și nestructurate.

Din punct de vedere tehnic, depășirea rigidității specifice aplicațiilor monolitice a determinat alegerea unei arhitecturi decuplate pe mai multe straturi, unde componenta de interfață este complet separată de cea responsabilă cu logica internă. Această abordare oferă flexibilitate în dezvoltare, permițând modificarea regulilor de business fără a afecta stabilitatea interfeței grafice. Totodată, modul de randare modern din browser asigură o experiență de navigare fluidă, elementele grafice actualizându-se fără reîmprospătarea întregii pagini, ceea ce crește productivitatea operatorilor și reduce timpul alocat sarcinilor repetitive.

Creșterea gradului de transparență și oferirea unui control accesibil clientului final reprezintă o motivație centrală a proiectului. Prin intermediul unui panou de control interactiv, utilizatorul poate urmări de la distanță fiecare etapă de evoluție a dosarului său, fiind informat atunci când vehiculul intră în faza de analiză, de aprobare financiară sau de reparație efectivă. Această vizibilitate reduce incertitudinea specifică metodelor administrative clasice și scade volumul de muncă de la birourile de asistență, transformând o procedură gestionată tradițional într-un flux de lucru mai predictibil și mai eficient pentru toate părțile implicate.

Capitolul 3. Tehnologii utilizate în dezvoltarea aplicației

3.1. Limbajul de programare Java și utilizarea acestuia în dezvoltarea aplicațiilor enterprise

Caracterul puternic tipizat și orientarea pe obiecte definesc limbajul de programare Java ca o opțiune optimă în vederea construirii unor sisteme software complexe, sigure și independente raportat la platforma hardware subiacentă. Printr-un mecanism funcțional hibrid care îmbină armonios compilarea și interpretarea, codul sursă suferă o transformare inițială într-un format intermediar denumit bytecode, urmând ca acesta să fie pus în execuție de Mașina Virtuală Java pe stația gazdă [7]. Portabilitatea aplicației este astfel garantată, în timp ce administrarea centralizată a execuției la nivelul serverului favorizează transmiterea rezultatelor către utilizator într-o manieră predictibilă și riguros controlată.

Versatilitatea evidentă în dezvoltarea sistemelor informatice de tip enterprise reprezintă unul dintre principalele avantaje ale ecosistemului Java. Regulile stricte asociate tipizării, alături de structura robustă a limbajului, determină adoptarea sa frecventă de către inginerii care proiectează arhitecturi critice, contexte în care stabilitatea și siguranța codului devin imperative. Tehnologia poate fi utilizată cu succes atât pentru implementarea unor componente izolate, cât și în cazul platformelor de mari dimensiuni, acoperind aplicații backend de mare complexitate, servicii web distribuite ori sisteme dedicate procesării unor volume masive de date. Merită menționat faptul că Java își dovedește utilitatea și ca soluție open-source prin intermediul unor distribuții comunitare precum OpenJDK [7], aspect ce facilitează integrarea sa în infrastructuri software extinse fără a genera costuri financiare legate de licențiere. O comunitate globală extrem de activă susține evoluția continuă a acestui limbaj, dinamica fiind asigurată de un ciclu predictibil de lansare a noilor versiuni la un interval de șase luni.

Integrarea strânsă cu numeroase ecosisteme și framework-uri industriale le permite programatorilor să aplice concepte de inversiune a controlului și injectare a dependențelor direct în straturile logice fundamentale, simplificând crearea de servicii web dinamice și securizate. Standardizarea proceselor de dezvoltare devine mai facilă datorită gamei extinse de API-uri și biblioteci puse la dispoziție prin inițiative de profil cum este Jakarta EE. Suportul nativ conceput pentru interacțiunea cu bazele de date relaționale prin intermediul specificațiilor JPA completează aceste avantaje, oferind posibilitatea utilizării unor framework-uri de tip ORM, un exemplu elocvent fiind Hibernate, cu scopul de a stoca, accesa și mapa eficient seturile de date printr-o abstractizare completă la nivelul stratului de persistență.

3.2. Framework-uri și tehnologii pentru dezvoltarea backend-ului

Dezvoltarea unor sisteme software din categoria enterprise reclamă o fundație tehnologică caracterizată prin rigoare și stabilitate. Sarcinile repetitive legate de securitate, conectivitate sau managementul tranzacțiilor sunt preluate eficient de framework-urile moderne, fapt ce simplifică semnificativ activitatea programatorilor. Punerea la dispoziție a unor structuri pretestate sporește

gradul de siguranță al întregii platforme informatice. Ecosistemul Spring Boot a fost selectat special pentru componenta de backend a acestei aplicații, această alegere tehnică asigurând o viteză optimă în execuție și o modularizare ideală a tuturor funcționalităților implementate.

3.2.1. Arhitectura aplicațiilor web bazate pe Java

Arhitectura reprezintă structura de rezistență a oricărui sistem informatic complex. În cazul aplicațiilor web dezvoltate în limbajul Java, standardul industrial actual impune decuplarea strictă a componentelor software. Aplicația realizată respectă acest principiu prin adoptarea unei arhitecturi stratificate, în care fiecare nivel are o responsabilitate unică și bine definită. Separarea codului previne apariția erorilor în evoluția aplicației. Legăturile dintre straturile de prezentare și logica de business sunt gestionate automat de framework prin mecanismul de Inversiune a Controlului.

Punctul de pornire al întregii arhitecturi se bazează pe configurarea automată a serverului și scanarea pachetelor de cod. Aplicația rulează un server web embedded. Nu mai este necesară configurarea unui server extern dedicat. Aceasta abordare izolează aplicația într-un container autonom, gata de execuție. Arhitectura stratificată asigură o mentenanță facilă pe termen lung. Modificarea logicii dintr-un strat nu afectează funcționarea celorlalte niveluri ale sistemului.

3.2.2. Implementarea serviciilor REST

Fluiditatea sistemului operațional depinde în mod vital de comunicarea stabilă între serverul Java și interfața React.js. Datorită simplității lor native, serviciile construite conform stilului arhitectural REST s-au impus ca un standard incontestabil în ingineria software contemporană [8]. Schimbul rapid de date este facilitat de faptul că serverul nu mai generează interfețe vizuale complete, ci furnizează exclusiv date brute structurate în format JSON. Fiecare acțiune prin care utilizatorul interacționează cu datele se mapează direct pe metodele specifice protocolului HTTP.

Cererile asincrone provenite din frontend sunt interceptate de rutele expuse de către componentele de control. Datele care sosesc prin rețea sunt preluate și transformate automat din format text în obiecte gata de a fi manipulate de logica internă a aplicației. Blocarea intrărilor eronate este asigurată prin validarea instantanee a integrității datelor direct la recepție. Clasa responsabilă de răspuns garantează un control total asupra fluxului informațional, returnând obiectul procesat împreună cu codurile de status HTTP specifice. Acest model oferă o interacțiune predictibilă și sigură între toate componentele care alcătuiesc sistemul informatic.

3.3. Biblioteca React.js pentru dezvoltarea interfețelor utilizator

Dezvoltată și menținută de Meta în colaborare cu o comunitate activă de ingineri software [9], React.js reprezintă o bibliotecă JavaScript orientată pe componente, recunoscută drept unul dintre cele mai utilizate instrumente destinate construirii unor interfețe utilizator dinamice. Modul în care datele sunt propagate către ecran este profund transformat de paradigma sa declarativă, dezvoltatorul specificând modul în care ar trebui să arate interfața în funcție de starea curentă, în

timp ce biblioteca gestionează autonom actualizările necesare, renunțându-se la descrierea pașilor expliți de modificare a interfeței. Redundanța codului este diminuată și mentenanța proiectelor de mari dimensiuni devine mai simplă prin descompunerea interfețelor complexe în componente izolate și reutilizabile. Fiecare componentă își administrează propria stare internă, definindu-și structura vizuală cu ajutorul JSX, o extensie sintactică concepută să combine capacitățile JavaScript cu expresivitatea HTML în cadrul aceluiași fișier. Această abordare modulară participă direct la comprimarea timpului de dezvoltare în cazul aplicațiilor de tip Single Page Application, unde fluiditatea interacțiunii atinge un nivel comparabil cu cel al unui software executat local.

Mecanismul Virtual DOM reprezintă elementul central care conferă bibliotecii React.js o performanță remarcabilă, manipularea directă a arborelui paginii din browser fiind înlocuită cu operații rulate asupra unei replici virtuale stocate în memoria clientului. Modificările frecvente aduse DOM-ului real în cadrul arhitecturilor web tradiționale generează operații costisitoare de reflow și repaint, fenomene care afectează vizibil fluiditatea generală. React evită aceste inconveniente prin intermediul unei comparări matematice a stării curente a arborelui virtual cu versiunea precedentă, proces realizat printr-un algoritm de diferențiere denumit reconciliere [10]. Structura reală din browser integrează exclusiv fragmentele modificate, optimizând procesul de randare și eliminând complet reîncărcările inutile ale paginii web.

Tranziția facilitată de introducere a mecanismelor de tip Hooks a determinat arhitectura modernă a aplicațiilor React să favorizeze utilizarea componentelor funcționale în detrimentul celor bazate pe clase. API-uri precum useState, destinat gestionării stării locale, și useEffect, utilizat pentru orchestrarea efectelor secundare și corelarea cu ciclul de viață al componentelor [9], contribuie la reducerea complexității codului și la simplificarea testării unitare. Logica devine astfel mult mai compactă și mai facil de urmărit, fiind simplificată corelarea directă dintre stările interfeței grafice și datele recepționate de la server.

3.4. Sistemul de gestiune a bazelor de date MySQL

MySQL este un sistem de gestiune a bazelor de date relaționale cu sursă deschisă [11], utilizat pe scară largă atât în aplicații web de dimensiuni medii, cât și în infrastructuri de nivel enterprise. Datele sunt organizate în tabele bidimensionale, iar relațiile logice dintre acestea sunt definite prin constrângeri de integritate operaționalizate prin chei primare și chei externe. Modelarea corectă a schemelor relaționale reprezintă o etapă esențială în proiectarea oricărei aplicații care lucrează cu date structurate, prevenind anomalii de inserție, ștergere sau actualizare și asigurând o propagare coerentă a informațiilor spre nivelul backend.

În contextul aplicațiilor cu cerințe ridicate de fiabilitate, MySQL se justifică prin conformitatea sa nativă cu proprietățile ACID: atomicitate, consistență, izolare și durabilitate [11]. Acest set de proprietăți garantează că tranzațiile sunt executate complet sau deloc, eliminând stările intermediare ambigue și protejând integritatea datelor în fața unor situații neprevăzute precum erori de rețea sau întreruperi bruște ale sistemului. Fiecare operațiune modifică starea bazei de date doar dacă respectă toate regulile predefinite, ceea ce face MySQL o alegere potrivită pentru aplicații în care corectitudinea datelor este critică.

Performanța și flexibilitatea operațională sunt asigurate de motorul de stocare InnoDB [11], care implementează blocare la nivel de rând, o caracteristică importantă pentru optimizarea accesului concurent într-un mediu cu trafic ridicat. InnoDB gestionează automat constrângerile referențiale și utilizează structuri de indexare de tip B-Tree pentru interogări rapide pe volume mari de date, minimizând numărul de accesări ale discului fizic și reducând timpul de răspuns pentru operațiunile complexe de filtrare și selecție.

În arhitectura backend a aplicației, eficiența acestor mecanisme de indexare influențează direct rata de răspuns transmisă clientului React. Deși MySQL poate ridica provocări legate de scalabilitatea orizontală la volume foarte mari de date și necesită o configurare atentă a pool-ului de conexiuni, sistemul oferă o soluție robustă și matură, bine adaptată cerințelor unui proiect de complexitate medie spre ridicată.

3.5. Comunicarea între componentele aplicației prin API REST

Comunicarea dintre componentele unei aplicații distribuite moderne se bazează frecvent pe stilul arhitectural REST [6], un model care nu reprezintă un protocol rigid, ci un set de constrângeri menite să asigure o decuplare clară între interfața utilizatorului și logica de business stocată pe server. Principiul central al acestei paradigme este absența stării, cunoscut sub denumirea de statelessness: fiecare solicitare trimisă de client trebuie să conțină toate informațiile necesare procesării sale independente, fără ca serverul să rețină vreun context al sesiunii anterioare. Această caracteristică simplifică scalabilitatea orizontală a sistemului și crește toleranța la erori, deoarece fiecare cerere poate fi procesată independent de celelalte.

Protocolul HTTP este utilizat pentru realizarea acestor interacțiuni, iar comunicarea se desfășoară asincron pentru a evita blocarea interfeței grafice în așteptarea răspunsului. Solicitățile sunt direcționate către adrese specifice denumite endpoint-uri, iar semantica fiecărei operațiuni este definită prin verbele HTTP standardizate, corespunzătoare operațiilor CRUD. Metoda GET este rezervată interogării și extragerii de resurse fără efecte secundare asupra bazei de date. POST introduce date noi și inițiază crearea de entități pe server. PUT și PATCH controlează actualizarea integrală sau parțială a resurselor existente, iar DELETE comandă ștergerea elementelor vizate. Răspunsul serverului este codificat prin coduri de stare HTTP: clasa 2xx semnalează succesul operațiunii, clasa 4xx indică erori generate de client, legate de sintaxă sau autorizare, iar clasa 5xx semnalează erori interne ale serverului backend.

Formatul utilizat pentru schimbul de date între componentele aplicației este JSON, un standard de serializare bazat pe o sintaxă simplă de tip cheie-valoare, interpretat nativ de motorul JavaScript din browser și mapat automat în obiecte Java de către Spring Boot. Amprenta de rețea a mesajelor JSON este redusă comparativ cu formatele mai vechi, ceea ce contribuie la fluiditatea comunicării. Atunci când componentele aplicației rulează pe porturi sau domenii distincte, browserele activează mecanismul Same-Origin Policy, care blochează implicit cererile provenite din surse externe. Pentru a permite interoperabilitatea securizată, configurarea politicii CORS devine necesară: serverul Spring Boot specifică explicit prin headere HTTP suplimentare care origini externe au permisiunea de a accesa resursele expuse prin API. Absența unei configurări

corecte a CORS determină respingerea automată a cererilor la nivelul browserului, întrerupând comunicarea dintre frontend și backend. Implementarea corectă a acestor headere asigură un flux informațional securizat și funcțional, validând arhitectura ca un sistem decuplat și ușor de extins.

3.6. Avantajele utilizării arhitecturii Java – React.js – MySQL

Combinarea limbajului Java, gestionat de Spring Boot pe stratul de backend, cu biblioteca React.js pe partea de frontend și sistemul relational MySQL pentru persistența datelor configurează o arhitectură clasică de tip 3-Tier [12], un model pe trei straturi considerat un standard de maturitate în ingineria software actuală. Această decizie de design asigură o separare clară a responsabilităților prin izolarea interfeței grafice de mecanismele interne de procesare logică și de structurile de stocare, ceea ce protejează codul împotriva degradării progresive sub presiunea extinderilor ulterioare și oferă o bază stabilă pentru sisteme de complexitate ridicată.

Această arhitectură constă în decuplarea dintre client și server. Deoarece React.js rulează în browserul utilizatorului ca o aplicație de tip Single Page Application, iar nucleul Java își execută logica pe server expunând endpoint-uri REST, cele două subsisteme pot parcurge cicluri separate de dezvoltare, testare și scalare. Schimbul de date se realizează asincron, prin mesaje serializate în format JSON, ceea ce reduce consumul de lățime de bandă și permite serverului Java să aloce resursele disponibile evaluării regulilor de business și gestionării securității, în timp ce clientul preia responsabilitatea randării interfeței grafice.

Tipizarea strictă a Java permite interceptarea erorilor logice încă din faza de compilare și oferă mecanisme clare de tratare a excepțiilor, contribuind la protejarea infrastructurii de backend împotriva unor vulnerabilități frecvente în mediul enterprise. La nivelul persistenței, MySQL asigură consistența relațiilor prin tranzacții conforme cu standardul ACID, prevenind coruperea sau pierderea înregistrărilor în scenarii de acces simultan. La nivelul prezentării, React.js include un mecanism nativ de escapare automată (transformarea automată a caracterelor speciale în reprezentări sigure pentru afișarea în paginile Web) a caracterelor în JSX, care neutralizează atacurile de tip Cross-Site Scripting fără a necesita intervenții manuale suplimentare din partea dezvoltatorului.

În ceea ce privește scalabilitatea, toate cele trei tehnologii dispun de comunități active la nivel global, documentații extinse și instrumente mature de testare automatizată, ceea ce face din această combinație o alegere tehnologică viabilă pe termen lung. O aplicație construită pe această arhitectură poate evolua de la o structură monolitică inițială spre un model de microservicii containerizate, adaptându-se în timp la creșterea numărului de utilizatori și la volumele tot mai mari de date procesate.

Capitolul 4. Analiza și proiectarea sistemului informatic

4.1. Identificarea cerințelor funcționale ale sistemului

Stabilirea cerințelor funcționale constituie punctul de pornire în dezvoltarea oricărei platforme informatice eficiente. Aceste specificații detaliază comportamentul pe care sistemul trebuie să îl aibă și interacțiunile directe ale utilizatorilor cu modulele de program create. Rolul lor fundamental este de a clarifica acțiunile rulate de aplicație. În cadrul unui centru de gestionare a daunelor auto, varietatea activităților zilnice impune o delimitare clară a drepturilor de acces și a operațiunilor permise pentru fiecare utilizator în parte.

Existența unui modul sigur și simplu de utilizat pentru autentificare și autorizare este de o importanță deosebită. Sistemul oferă utilizatorilor posibilitatea de a-și crea conturi individuale și de a accesa datele protejate în deplină siguranță. Restricționarea accesului la resurse se bazează strict pe rolul salvat în baza de date. Un client obișnuit nu va avea niciodată permisiunea de a modifica valoarea devizului stabilit de un inspector, iar un mecanic auto nu poate introduce dosare noi în numele unei societăți de asigurări.

Pentru administrarea corectă a elementelor din sistem, aplicația integrează operațiuni complete de adăugare, vizualizare, modificare și ștergere pentru clienți și autovehicule. Utilizatorii înregistrează serii de șasiu, numere de înmatriculare și specificații tehnice valide. Modulul destinat managementului dosarelor reprezintă nucleul funcțional al întregii platforme. Aplicația preia imagini cu avariile descoperite, le asociază unui dosar unic și permite actualizarea în timp real a stării autoturismului.

Obținerea rapoartelor aduce o utilitate majoră în activitatea zilnică a companiei. La finalizarea inspecției, aplicația adună datele tehnice și exportă un document oficial sub formă de fișier PDF, acesta fiind folosit ca notă de constatare. Toate interacțiunile menționate se desfășoară continuu, asigurând un parcurs fluid din momentul introducerii datelor până la eliberarea documentului final.

4.2. Identificarea cerințelor nefuncționale

Evaluarea unui sistem informatic nu se limitează la ceea ce acesta poate executa, motiv pentru care cerințele nefuncționale sunt esențiale pentru stabilirea criteriilor de calitate, a constrângerilor tehnice și a standardelor de securitate. Toate aceste criterii influențează direct deciziile de arhitectură software și condiționează funcționarea optimă a platformei în condiții de exploatare reală, cu un volum ridicat de date eterogene.

Pentru aplicația web dezvoltată, atributele de calitate au fost identificate și structurate pe categorii clare, după cum urmează.

- Performanță și timp de răspuns. Timpul de răspuns al interfețelor de programare pentru operațiunile uzuale, cum ar fi filtrarea dosarelor sau încărcarea listelor, nu trebuie să depășească 500 de milisecunde în condiții normale de utilizare. Interfața utilizator trebuie să asigure o randare fluidă a elementelor grafice prin optimizarea

componentelor din memorie, eliminând necesitatea reîncărcării complete a paginilor la fiecare interacțiune.

- **Securitate și autorizare.** Stocarea parolelor în baza de date se realizează prin aplicarea unor algoritmi de criptare cu cheie unică, iar schimbul de informații între interfață și serverul de backend este securizat prin jetoane de acces temporare, garantând restricționarea datelor în conformitate cu rolul deținut de fiecare utilizator.
- **Gestionarea concurenței.** Sistemul trebuie să suporte interacțiunea simultană a mai multor utilizatori fără a genera blocaje la nivelul resurselor hardware sau al fluxurilor de date, comportament controlat prin mecanisme de gestionare a bazinelor de conexiuni și prin izolarea tranzacțiilor la nivelul bazei de date.
- **Uzabilitate și responsivitate.** Interfața grafică este proiectată pentru a fi complet adaptabilă pe diverse rezoluții, oferind inspectorilor posibilitatea de a opera eficient de pe tablete în teren, iar clienților opțiunea de a încărca documente direct de pe dispozitive mobile, pe baza unui design curat și intuitiv.
- **Decuplare și mentenabilitate.** Modificările aduse logicii interne din zona de backend sau schimbarea unor reguli administrative nu trebuie să afecteze structura sau stabilitatea interfeței grafice. Această independență este garantată prin utilizarea unei arhitecturi stratificate și a unor formate standardizate pentru transferul de date.
- **Persistență și tranzaționalitate.** Toate operațiunile care implică modificări financiare, adăugări de piese sau aprobări tehnice trebuie să respecte principiile de consistență ale bazelor de date. În cazul în care salvările intermediare eșuează din motive tehnice, sistemul execută o revocare completă a pașilor parcurși, prevenind stocarea unor date fragmentate sau incorecte.

Atingerea unui timp de răspuns optim este condiționată de deciziile de proiectare luate la nivelul fiecăruia dintre cele trei straturi principale ale aplicației. La nivelul interfeței utilizator, structura grafică este concepută ca o aplicație bazată pe o singură pagină, ceea ce înseamnă că elementele vizuale nu se reîncarcă complet la fiecare navigare, ci sunt actualizate selectiv doar acolo unde starea s-a modificat, reducând la minimum utilizarea resurselor din browser. La nivelul serverului de aplicații, cererile primite sunt tratate asincron, iar un bazin de conexiuni permanente către baza de date elimină timpul necesar deschiderii unor noi canale de comunicare la fiecare solicitare, scăzând latența generală. La nivelul bazei de date, structura tabelor a fost optimizată prin adăugarea de indecși pe coloanele frecvent utilizate în interogări, cum sunt numărul dosarului sau stadiul acestuia, prevenind scanarea completă a datelor și menținând stabilitatea platformei pe măsură ce volumul înregistrărilor crește.

4.3. Arhitectura generală a aplicației

Structura generală a sistemului propus urmează modelul unei arhitecturi pe trei straturi, o abordare clasică bazată pe relația client-server. Acest mod de lucru asigură o separare clară între

interfața utilizatorului, logica de procesare și stocarea permanentă a informațiilor, oferind flexibilitate, securitate ridicată și o mentenanță simplă pentru întreaga platformă destinată managementului daunelor auto.

Principiul de bază al acestei organizări constă în împărțirea responsabilităților, fiecare strat funcționând ca o entitate independentă din punct de vedere tehnic. Comunicarea se realizează doar cu nivelurile direct învecinate prin protocoale standard, fără ca un strat să aibă nevoie de detalii despre implementarea internă a celuilalt.

Organizarea conceptuală cuprinde trei niveluri distincte. Stratul de prezentare constituie componenta de frontend care afișează interfața grafică, preia acțiunile utilizatorilor și trimite cererile asincrone către server, asigurând o redare dinamică. Stratul de logică, adică zona de backend, coordonează regulile de business, validează datele sosite, securizează punctele de acces prin jetoane temporare și controlează execuția operațiunilor din sistem. Stratul de persistență se ocupă de salvarea structurată a tuturor informațiilor în tabele relaționale, garantând integritatea și ordonarea eficientă a datelor prin indecși.

Schimbul de date între componente se face printr-o interfață de programare de tip REST, apelând protocolul HTTP și formatul JSON. Fluxul unei cereri parcurge pași tehnici bine definiți.

Totul începe cu inițierea cererii, moment în care utilizatorul interacționează cu elementele vizuale din browser, iar aplicația strânge datele într-un obiect structurat și trimite o cerere asincronă către server. Interceptarea și autorizarea reprezintă etapa următoare, în care serverul primește cererea prin controlerele dedicate, iar un strat de securitate verifică jetonul de acces din header pentru a confirma identitatea și drepturile utilizatorului înainte de procesare. Urmează procesare logică, fază în care informațiile ajung la serviciile interne unde se aplică regulile centrului de daune, cum ar fi calculul sumelor sau verificarea plafoanelor de plată. Salvarea datelor se face atunci când componenta de business apelează stratul de persistență, acesta transformând obiectele din cod în instrucțiuni SQL executate în condiții de siguranță în baza de date. Transmiterea răspunsului încheie acest circuit, serverul trimițând un cod de stare către browser, care își actualizează starea și afișează modificările pe ecran fără a reîncărca pagina.

Această structură stratificată protejează sistemul de erori și izolează componentele. Dacă interfața grafică este înlocuită cu o aplicație mobilă sau modificată, logica de pe server și organizarea bazei de date nu sunt afectate, fapt ce confirmă flexibilitatea soluției propuse.

4.4. Modelarea sistemului informatic

4.4.1. Diagrama cazurilor de utilizare

Diagrama cazurilor de utilizare definește interacțiunile dintre actorii externi și funcțiile expuse de aplicație, stabilind granițele logice ale sistemului informatic. Pentru o platformă dedicată managementului operațional dintr-un centru de daune auto, complexitatea fluxurilor impune o delimitare strictă a rolurilor. Au fost identificați cinci actori principali, fiecare având drepturi și responsabilități bine definite în sistem, așa cum este exemplificat în Figura 4.1.

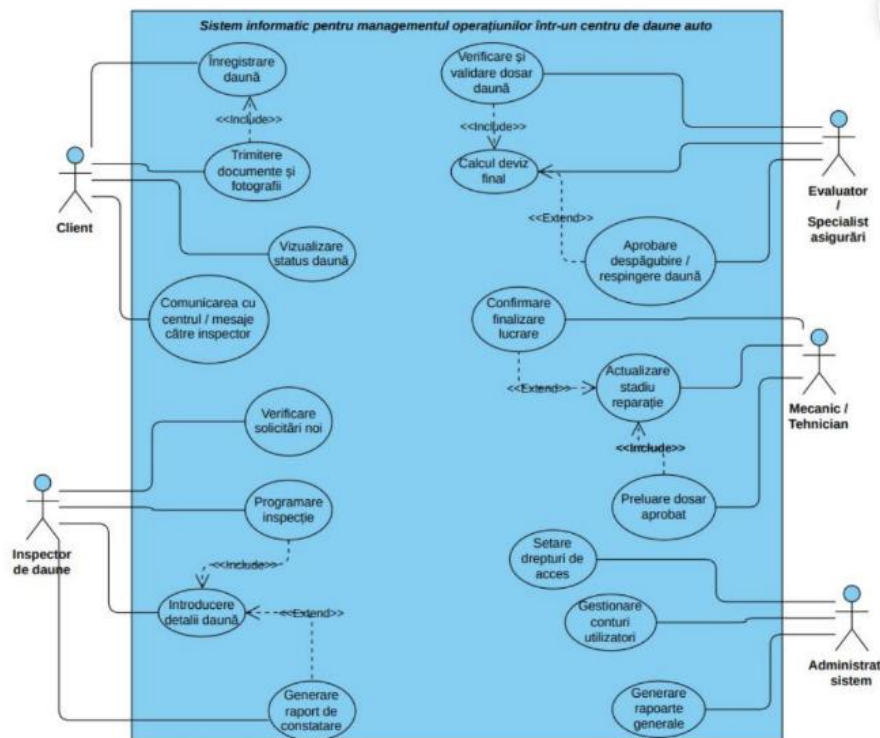


Figura 4.1. Scenariile de interacțiune și atribuțiile actorilor în cadrul sistemului.

Clientul reprezintă utilizatorul păgubit care inițiază procesul operațional. Acesta interacționează direct cu modulele de înregistrare a daunei și de trimitere a documentelor și fotografiilor, cazuri legate printr-o relație de incluziune de tip <<Include>>. De asemenea, clientul poate vizualiza statusul curent al dosarului și poate comunica prin mesaje cu personalul centrului. Inspectorul de daune are rolul de a verifica solicitările noi și de a programa inspecțiile tehnice, acțiune care include în mod obligatoriu introducerea detaliilor daunei. Procesul de generare a raportului de constatare rulează ca o extensie de tip <<Extend>> a acestui flux.

Evaluatorul sau specialistul în asigurări se concentrează pe aspectele financiare ale dosarului. Acesta verifică și validează cazul, operațiune ce include calculul devizului final, în timp ce aprobarea sau respingerea despăgubirii extinde decizia logică pe baza calculelor efectuate. Mecanicul sau tehnicianul din service intervine în faza de reparație a autovehiculului. El preia dosarul aprobat și actualizează stadiul lucrărilor, acțiune extinsă ulterior prin confirmarea finalizării reparației. Administratorul de sistem deține controlul global asupra platformei, fiind responsabil de setarea drepturilor de acces, gestionarea conturilor de utilizatori și generarea rapoartelor generale de performanță.

4.4.2. Diagrama componentelor sistemului

Modularizarea codului sursă și structura fizică a aplicației sunt descrise în detaliu prin diagrama de componente. Sistemul urmează un model Full-Stack decuplat, straturile software fiind independente și comunicând prin protocoale sigure de rețea. Această organizare asigură o

mentenanță ușoară și previne blocajele în faza de rulare, configurația arhitecturală fiind redată în Figura 4.2.

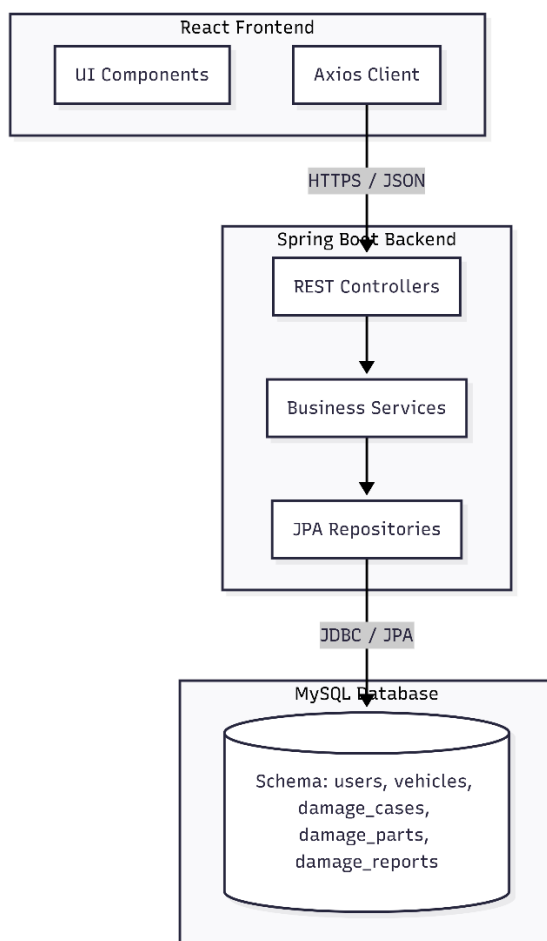


Figura 4.2. Organizarea componentelor software și protocoalele de legătură.

Stratul de prezentare constă în componenta React Frontend, care include elementele vizuale destinate utilizatorului și clientul de rețea Axios. Comunicarea cu serverul se face asincron, prin cereri securizate HTTPS și mesaje JSON. Nucleul central este reprezentat de pachetul Spring Boot Backend, structurat pe trei niveluri: controlere REST, servicii de business și repozitorii JPA, primele preluând cererile din interfață pentru a le trimite straturilor logice. Stratul de persistență se află la baza arhitecturii, unde repozitoriile JPA interacționează cu baza de date relațională MySQL printr-o conexiune JDBC securizată, asigurând salvarea corectă a datelor despre utilizatori, mașini și dosare active.

4.4.3. Diagrama fluxului de date

Transformările succesive ale datelor în timpul utilizării aplicației sunt mapate prin diagrama fluxului de date de Nivel 1. Modelul descrie mișcarea informațiilor între entitățile externe și procesele serverului, indicând locurile fizice de stocare, așa cum reiese din Figura 4.3.

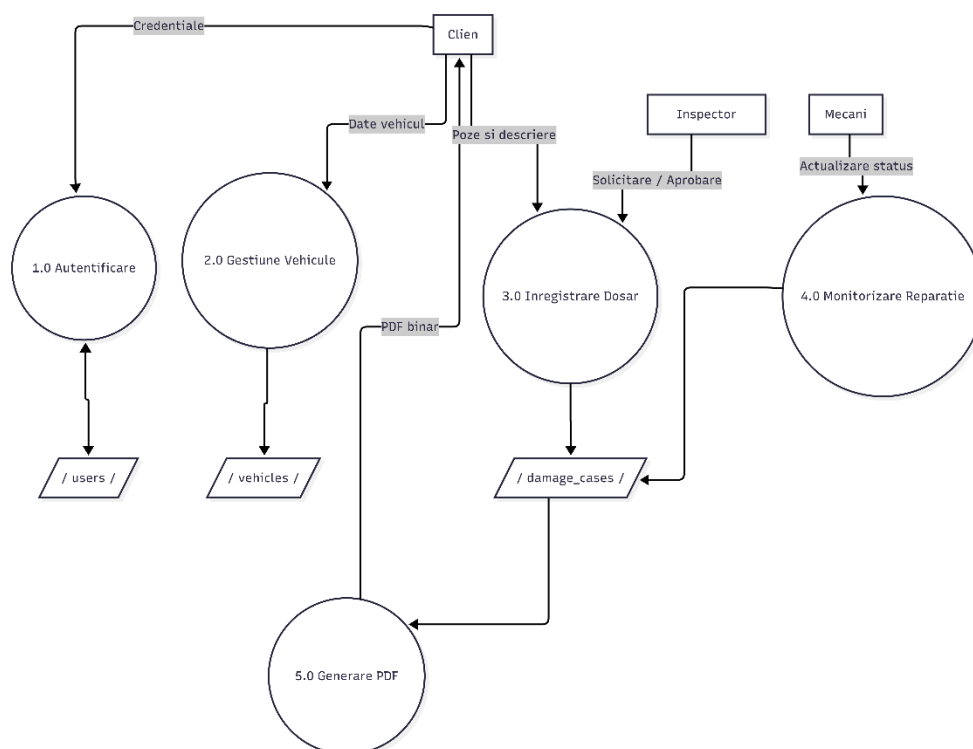


Figura 4.3. Diagrama de circulație și transformare a datelor

Fluxul este pornit de acțiunile utilizatorilor, clientul trimițând datele de conectare către procesul 1.0 Autentificare, care verifică informațiile în baza de date cu utilizatori. Pentru înregistrarea bunurilor, clientul transmite datele tehnice ale mașinii către procesul 2.0 Gestiune Vehicule, detaliile fiind salvate în tabela vehiculelor. Deschiderea unui dosar se face prin trimiterea imaginilor și a descrierilor către procesul 3.0 Înregistrare Dosar, acțiune care actualizează tabela cazurilor de daună și este urmărită de inspector prin cereri de aprobare.

În faza de remediere a avariilor, mecanicul introduce datele despre progres în procesul 4.0 Monitorizare Reparație, schimbând starea în tabela cazurilor de daună. Circuitul informațional de bază se încheie prin procesul 5.0 Generare PDF, care preia datele consolidate din tabela de daune, assemblează documentul oficial și livrează fișierul PDF binar direct în interfața clientului.

4.5. Proiectarea bazei de date

Salvarea și gestionarea eficientă a informațiilor reprezintă pilonul de susținere pentru orice aplicație de tip enterprise. Datele unui sistem destinat managementului daunelor auto trebuie organizate foarte bine, proiectarea bazei de date fiind realizată după un model relațional. S-a ales sistemul MySQL datorită stabilității sale recunoscute, acesta permițând definirea unor legături stricte între entități și prevenind erorile de inserție sau ștergere. Mecanismele native de indexare oferă o viteză bună de răspuns chiar și la creșterea volumului de date, totul fiind structurat logic pentru a sprijini fluxul de business.

4.5.1 Modelul conceptual al bazei de date

Cel mai înalt nivel de abstractizare a realității dintr-un centru de daune este reprezentat de modelul conceptual. Rolul său este de a defini entitățile de bază și de a trasa legăturile logice dintre ele, fără a coborî în detalii de sintaxă sau stocare fizică. Analiza a identificat șase entități principale, capabile să acopere toate nevoile aplicației.

Prima entitate este User, care definește componenta umană a sistemului și reunește toate categoriile de personal, de la administratori la clienți. Entitatea Vehicle reprezintă bunul material supus inspecției, legătura dintre persoană și mașină trebuind să fie clară. Nucleul sistemului este entitatea DamageCase, care descrie un eveniment, marcând deschiderea dosarului și coordonarea elementelor de timp și spațiu ale daunei. Pentru detalierea stricăciunilor fizice s-a definit entitatea DamagePart, care păstrează rezultatele analizei tehnice și măsoară severitatea loviturii. Fluxurile de asistență automată sunt gestionate prin entitățile Message și DamageReport, prima salvând dialogul din chat, iar a doua păstrând rapoartele create de inteligența artificială, formând împreună o rețea logică ce descrie complet viața unui dosar.

4.5.2 Modelul logic al bazei de date

Modelul logic traduce entitățile conceptuale într-o structură de tabele capabilă să fie implementată direct pe un server relațional. În această fază, accentul cade pe normalizarea datelor și pe stabilirea tipurilor corecte pentru fiecare atribut în parte. Procesul de normalizare a fost împins până la Forma Normală Trei (3NF). Scopul a fost eliminarea oricărei redundanțe funcționale. Astfel, nicio informație nu este salvată de două ori în locuri diferite. Acest lucru previne erorile grave la actualizarea datelor.

Definirea tipurilor de date a fost făcută cu o atenție deosebită pentru performanța pe termen lung. Toate cheile primare folosesc tipul BIGINT sau INT. Acest tip asigură spațiu suficient pentru milioane de înregistrări fără degradarea vitezei de căutare.

Pentru atributele textuale s-a ales standardul VARCHAR(255). El oferă flexibilitate pentru nume, adrese de e-mail sau serii de șasiu, optimizând în același timp spațiul de memorie. Pentru elementele cu volum mare și impredictibil, cum sunt mesajele sau răspunsurile AI, s-a preferat tipul TEXT. Acesta permite stocarea de paragrafe lungi fără restricții rigide.

Timpul este o altă variabilă critică în gestionarea daunelor. Câmpurile destinate marcării momentelor temporale folosesc tipul DATETIME(6). Această opțiune permite o precizie de până la microsecundă. Este ideală pentru ordonarea cronologică a mesajelor în chat sau pentru stabilirea momentului exact în care un dosar și-a schimbat statusul operațional.

4.5.3 Structura tabelelor și relațiile dintre acestea

Organizarea riguroasă a componentelor este reflectată de structura bazei de date, garantând integritatea informațiilor pe parcursul tuturor fluxurilor. Fiecare tabel are un rol clar definit, iar legăturile stabilite prin chei străine se asigură că nicio înregistrare nu rămâne izolată. Întreaga structură de persistență, tipurile alese și legăturile sunt mapate în schema fizică generată, prezentată detaliat la finalul acestei secțiuni.

Tabelul `users` reprezintă baza securității și a controlului accesului, conținând cheia primară `id` de tip `BIGINT`, configurată cu incrementare automată în MySQL. Pentru identificare, tabelul folosește câmpurile `username` și `email`, ambele de tip `VARCHAR(255)`, în timp ce parola este stocată securizat în câmpul `password` prin hashing. Rolul este definit în câmpul `role` de tip `VARCHAR(50)`, determinând drepturile de navigare în interfață, iar pentru securitate extinsă sunt incluse câmpurile `reset_token` (`VARCHAR(255)`) și `reset_token_expiry` (`DATETIME(6)`).

Tabelul `vehicles` gestionează mașinile înregistrate de clienți, cheia primară fiind `id` de tip `BIGINT`. Caracteristicile sunt descrise prin coloanele `brand`, `model`, `plate_number` și `vin`, toate mapate ca `VARCHAR(255)`. Între tabelele `users` și `vehicles` există o relație de tip One-to-Many, deoarece un utilizator poate avea mai multe mașini în cont, dar o mașină are un singur proprietar, legătura fiind asigurată de cheia străină `user_id`.

Tabelul `damage_cases` coordonează logica dosarelor deschise, având identificatorul unic `id` de tip `BIGINT`. Data deschiderii este salvată în câmpul `creation_date` (`DATETIME(6)`), iar starea este urmărită prin `status` (`VARCHAR(255)`). Acest tabel adună informații din mai multe surse, având două chei străine esențiale: `user_id` pentru legătura cu clientul și `vehicle_id` pentru mașina implicată, ambele fiind relații Many-to-One.

Tabelul `damage_parts` detaliază constatările tehnice, cheia primară fiind `id` de tip `BIGINT`. Denumirea piesei este salvată în `part_name`, iar gravitatea în `severity`, ambele fiind `VARCHAR(255)`. Scorul de certitudine oferit de analizele automate este stocat în coloana `confidence_score` ca tip `DOUBLE`, iar relația Many-to-One cu tabelul `damage_cases` este realizată prin cheia străină `case_id`, un dosar putând avea mai multe piese avariate.

Tabelul `damage_reports` păstrează istoricul rapoartelor generate, având cheia primară `id` de tip `BIGINT`. Datele textuale sunt salvate în câmpul `description` de tip `TEXT`, identificarea mașinii se face prin `license_plate` (`VARCHAR(255)`), starea este urmărită prin `status` (`VARCHAR(30)`), iar data creării este salvată în `created_at` (`DATETIME`). Relația One-to-Many cu utilizatorii este realizată prin cheia străină `user_id`, structura completă fiind redată în Figura 4.4.

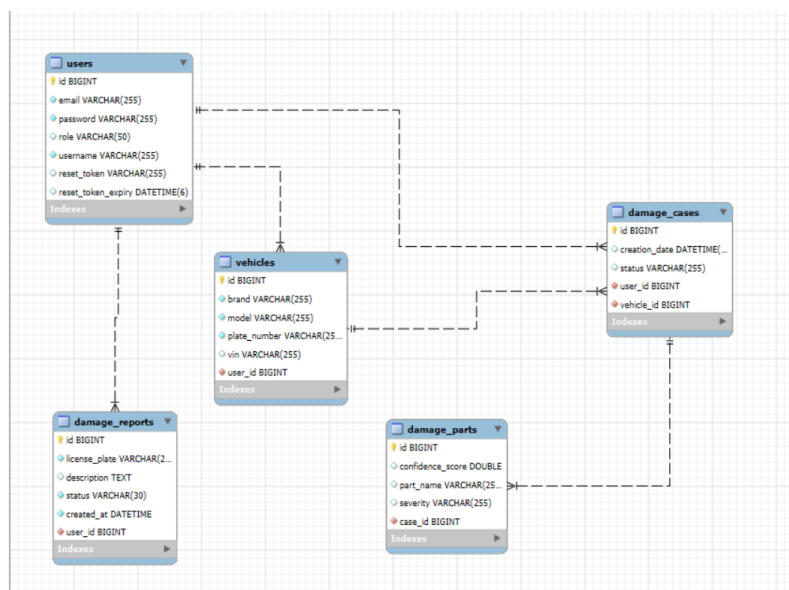


Figura 4.4. Arhitectura modelului relațional și structura fizică a bazei de date.

4.6. Definirea rolurilor utilizatorilor în sistem

4.6.1. Administrator

Acest rol deține drepturi complete de gestionare și răspunde de administrarea tehnică a platformei. Administratorul nu intervine în evaluarea sau reparația mașinilor, ci asigură structura logică necesară pentru activitatea celorlalți utilizatori. Atribuțiile sale cuprind managementul conturilor, proces ce implică crearea de utilizatori noi pentru operatori, inspectori sau service-uri, alocarea rolurilor și dezactivarea profilurilor inactive. El se ocupă de configurarea nomenclatoarelor globale, administrând datele folosite de aplicație, precum mărcile auto, societățile de asigurare și cataloagele de piese, urmărind totodată jurnalele de securitate pentru a asigura funcționarea continuă a platformei.

4.6.2. Operator centru de daune

Primul punct de contact intern cu un dosar este reprezentat de operatorul centrului de daune, a cărui misiune constă în preluarea, verificarea și înregistrarea notificărilor primite pentru a porni fluxul aplicației. În prima fază, operatorul analizează datele introduse de clienți și verifică valabilitatea polițelor de asigurare. Dacă descoperă neconcordanțe, el poate modifica sau completa detaliile tehnice ale vehiculelor, precum seria de șasiu, numărul de înmatriculare și datele proprietarilor. Etapa se încheie cu alocarea cazului, acțiune ce presupune generarea unui număr unic de dosar și trimiterea acestuia către un inspector disponibil, starea cazului fiind schimbată în mod corespunzător.

4.6.3. Inspector de daună

Inspectorul de daună ocupă un rol central în evaluarea tehnică și financiară a avariilor produse, utilizând aplicația pentru a analiza aspectele de conformitate și pentru a lua decizii cu caracter tehnic și economic pe baza polițelor de asigurare active. Responsabilitățile sale încep cu evaluarea digitală a avariilor, care presupune analizarea fotografiilor încărcate în dosar și completarea procesului-verbal de constatare direct în platformă. Ulterior, inspectorul stabilește soluțiile tehnice prin selectarea reperelor afectate și specificarea acțiunii necesare pentru fiecare piesă, optând între reparație sau înlocuire completă. Faza finală implică avizarea financiară a devizelor: inspectorul analizează costurile introduse de service-ul auto, aprobă sau respinge sumele propuse, stabilește plafonul maxim de despăgubire și mută dosarul în starea de aprobare pentru reparație.

4.6.4. Personal service auto

Execuția fizică a reparațiilor reprezintă responsabilitatea personalului din service-ul auto. Accesul acestui actor este limitat strict la modulele ce vizează vehiculele alocate unității proprii, vizualizarea dosarelor de la alte service-uri fiind blocată. Activitatea pornește de la preluarea listei de constatare, ce permite vizualizarea pieselor și a soluțiilor tehnice aprobate de inspector. Pe baza acestor detalii, personalul introduce devizul completând costurile pentru manoperă și prețurile

pieselor de schimb. În timpul lucrărilor, stadiul reparației este actualizat în timp real prin alegerea unor stări specifice, precum așteptarea pieselor, vopsitorie sau reparație finalizată, acțiune care informează automat clientul cu privire la evoluția cazului.

4.6.5 Client

Clientul, definit ca proprietar al vehiculului, asigurat sau păgubit, reprezintă utilizatorul extern al sistemului informatic. Introducerea acestui rol ca actor activ în aplicație are scopul de a digitaliza interacțiunea cu asiguratorul, eliminând birocrăția. Nivelul de acces al acestui utilizator este restricționat prin mecanisme de identificare care îi permit să acceseze exclusiv propriile date și dosare. Principala funcționalitate disponibilă este notificarea online a daunei, care permite inițierea unui dosar prin completarea unui formular digital și încărcarea fotografiilor cu avariile vehiculului și a documentelor personale. Ulterior, utilizatorul poate urmări evoluția cazului printr-un panou de bord personal, unde are acces în timp real la statusul dosarului și la stadiul în care se află vehiculul în service, asigurând astfel o trasabilitate completă a procesului.

4.7. Proiectarea interfețelor utilizator

Crearea interfețelor grafice a avut la bază dorința de a oferi fiecărui tip de utilizator un mediu de lucru adaptat sarcinilor sale, fără a-l expune la opțiuni irelevante sau la informații protejate. Claritatea vizuală combinată cu simplitatea funcțională a ghidat această etapă, fiecare pagină permițând utilizatorului să înțeleagă rapid acțiunile necesare și stadiul procesului. Deoarece aplicația folosește o structură de tip Single Page Application realizată în React.js, trecerea de la o secțiune la alta se face fără reîncărcarea paginii, asigurând o navigare fluidă.

Interfața administratorului este organizată în jurul unui panou de control ce oferă o privire de ansamblu asupra sistemului. Din această zonă se accesează modulul de management al utilizatorilor, unde se pot crea conturi, modifica date existente, alocă roluri sau dezactiva profiluri. Un alt modul este dedicat nomenclatoarelor, unde se adaugă sau se editează mărcile auto, asiguratorii parteneri și cataloagele de piese. Ecranul este conceput pentru a reduce numărul de pași, folosind formulare clare și confirmări vizuale rapide.

Gestionarea eficientă a dosarelor aflate în diverse stadii reprezintă scopul interfeței operatorului. Ecranul principal conține o lăsa a notificărilor primite, cu opțiuni de filtrare după stare, dată sau număr de dosar. La selectarea unui caz, operatorul deschide un formular de validare structurat pe secțiuni care îl ajută să verifice datele mașinii, polițele și documentele atașate. După validare, el accesează o zonă de alocare unde vede inspectorii disponibili și distribuie cazul prin câteva acțiuni simple, modificările fiind reflectate imediat.

Interfața inspectorului este construită în jurul dosarelor repartizate, punând accent pe accesul rapid la detaliile tehnice. La deschiderea unui caz, inspectorul folosește un vizualizator de imagini integrat și câmpurile destinate procesului-verbal. Alegerea pieselor afectate se face dintr-un nomenclator predefinit, specificând dacă soluția este reparația sau înlocuirea. Ulterior, aceeași zonă este utilizată pentru analiza devizului primit de la service, oferind posibilitatea de a adăuga

observații, de a schimba valori și de a aproba sau respinge costurile înainte de a trimite dosarul mai departe.

Afișând doar vehiculele repartizate unității proprii, interfața personalului de service este cea mai restrictivă. Ecranul principal arată mașinile aflate în lucru și starea fiecăreia. La selectarea unui vehicul, personalul vede piesele aprobate de inspector, introduce devizul cu prețurile detaliate și actualizează progresul alegând starea potrivită din listă, această schimbare fiind trimisă automat către operator și client.

Concepută pentru persoane fără cunoștințe tehnice intense, interfața clientului este cea mai simplă. La autentificare, clientul ajunge direct în panoul personal unde vede starea dosarelor deschise prin intermediul unor etape vizuale clare. El are la dispoziție un formular de notificare a daunei pentru a deschide un caz nou, completând detalii despre mașină, accident și atașând fotografiile necesare. Ecranul exclude orice element legat de alte dosare, informațiile fiind limitate la ceea ce este strict necesar, asigurând o experiență simplă și protecția deplină a datelor celorlalți utilizatori.

Capitolul 5. Implementarea aplicației

Transformarea conceptelor teoretice și a diagramelor de proiectare într-un produs software funcțional reprezintă etapa cea mai densă a întregului proiect. Acest capitol detaliază construcția efectivă a aplicației, analizând deciziile de programare, configurațiile adoptate și structura modulelor de cod. Implementarea s-a ghidat după principiul separării clare a responsabilităților. Toate componentele au fost realizate într-o manieră modulară pentru a facilita testarea și mentenanța viitoare a întregului sistem informatic.

5.1. Configurarea mediului de dezvoltare

Construcția efectivă a unui sistem informatic cu o arhitectură complexă solicită un mediu de lucru versatil și bine structurat. Pentru dezvoltarea componentei principale de backend și a interfeței grafice, s-a utilizat mediul de dezvoltare integrat IntelliJ IDEA Ultimate. Această suită industrială oferă un suport avansat atât pentru ecosistemul Java Spring Boot, cât și pentru tehnologiile web moderne bazate pe React.js. Centralizarea proiectului într-un singur IDE optimizează considerabil depănarea și managementul codului sursă. Gestionarea dependențelor de Java a fost realizată prin utilitarul Apache Maven, utilizând kitul de dezvoltare Java Development Kit (JDK) 17 ca versiune de bază pentru asigurarea stabilității tranzacțiilor.

Pe lângă tehnologiile principale, integrarea funcțiilor inteligente și a asistentului virtual a impus utilizarea unor componente specifice scrise în limbajul Python. Pentru dezvoltarea și testarea acestor module specifice de analiză textuală, s-a utilizat mediul de dezvoltare PyCharm. Acest IDE permite o gestionare strictă a scripturilor prin intermediul pachetelor sale native. Izolarea dependențelor și a librăriilor specifice de inteligență artificială s-a realizat prin configurarea unui mediu virtual de execuție (venv). Se previn astfel conflictele dintre pachetele software de sistem. Pentru persistența datelor, s-a configurat un server local MySQL Server, administrat vizual prin MySQL Workbench. Interconectarea tuturor modulelor este asigurată prin fișiere de proprietăți dedicate, ce definesc URL-urile și credențialele de acces necesare.

5.2. Implementarea componentei backend în Java

Implementarea componentei de backend reprezintă construirea motorului logic și operațional al întregii aplicații. Codul Java a fost organizat după o arhitectură stratificată, un tipar clasic ce previne amestecarea logicii de prezentare cu cea de persistență. Fiecare clasă a fost plasată într-un pachet specific (model, repository, service, controller), respectând cu strictețe responsabilitățile atribuite fiecărui nivel arhitectural.

5.2.1. Definirea modelelor de date

Transpunerea structurii relaționale a bazei de date în obiecte software manipulabile de către serverul Java s-a realizat prin intermediul entităților JPA. Fiecare clasă din pachetul model corespunde unei mese fizice din baza de date MySQL. Relațiile de tip One-to-Many și Many-to-

Unele au fost mapate utilizând adnotări specifice oferite de specificația Jakarta Persistence. Acest mecanism elimină necesitatea scrierii manuale a interogărilor de unire a tabelor.

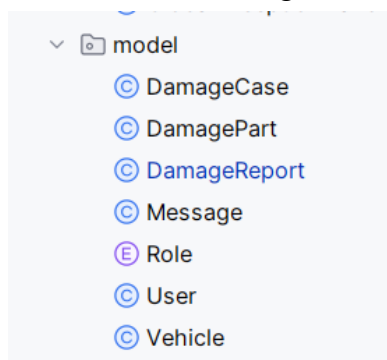


Figura 5.1. Structura pachetului model și organizarea claselor de tip entitate JPA

Pentru a exemplifica modul în care s-a realizat maparea și stabilirea legăturilor referențiale, se prezintă o parte reprezentativă din clasa de model Vehicle:

```
1 package ro.university.vehicledamagemanager.model;
2
3 > import ...
4
5
6
7
8
9 @Entity & Sorina
10 @Table(name = "vehicles")
11 @Getter
12 @Setter
13 @NoArgsConstructor
14 @AllArgsConstructor
15 public class Vehicle {
16
17     @Id
18     @GeneratedValue(strategy = GenerationType.IDENTITY)
19     private Long id;
20
21     @Column(nullable = false)
22     private String brand;
23
24     @Column(nullable = false)
25     private String model;
26
27     @Column(unique = true, nullable = false)
28     private String plateNumber;
29
30     @Column(unique = true)
31     private String vin;
32
33     @ManyToOne(fetch = FetchType.LAZY)
34     @JoinColumn(name = "user_id", nullable = false)
35     private User owner;
36 }
```

Figura 5.2. Fragment de cod privind maparea entității Vehicle și configurarea adnotărilor JPA.

După cum se poate observa în fragmentul de cod prezentat, adnotarea `@Entity` marchează clasa ca fiind o componentă persistentă. Conexiunea cu tabelul fizic este stabilită explicit prin `@Table(name = "vehicles")`. Proprietatea `vin` este definită ca fiind unică prin parametrul `unique = true`, interzicând astfel duplicarea aceleiași serii de șasiu în sistem. Legătura cu proprietarul mașinii este configurată prin `@ManyToOne`, utilizând strategia `FetchType.LAZY` pentru a optimiza consumul de memorie prin încărcarea datelor despre utilizator doar în momentul solicitării lor explicite. Cheia străină este mapată prin coloana `user_id`.

5.2.2. Implementarea logicii de business

Stratul de servicii înglobează toate regulile operaționale și algoritmi de manipulare a datelor din cadrul aplicației. Clasele din pachetul `service` acționează ca un izolator între endpoint-urile expuse și accesul direct la baza de date. Managementul fișierelor reprezintă o funcționalitate critică. Aplicația trebuie să salveze pozele cu avarii într-un mod securizat și ordonat, prevenind pierderile de date sau rescrierile accidentale.

Mai jos este prezentată implementarea metodei de stocare fizică a fișierelor din cadrul clasei `FileStorageService`:

```
1 package ro.university.vehicledamagemanager.service;
2
3 > import ...
11
12 @Service @Sorina
13 public class FileStorageService {
14
15     private final Path fileStorageLocation = Paths.get("uploads").toAbsolutePath().normalize(); // 2 usages
16
17     public FileStorageService() { @Sorina
18         try {
19             Files.createDirectories(this.fileStorageLocation);
20         } catch (IOException ex) {
21             throw new RuntimeException("Nu s-a putut crea directorul pentru incarcari.", ex);
22         }
23     }
24
25 @ public String storeFile(MultipartFile file) { no usages @Sorina
26     String fileName = UUID.randomUUID().toString() + "_" + file.getOriginalFilename();
27     try {
28         Path targetLocation = this.fileStorageLocation.resolve(fileName);
29         Files.copy(file.getInputStream(), targetLocation, StandardCopyOption.REPLACE_EXISTING);
30         return fileName;
31     } catch (IOException ex) {
32         throw new RuntimeException("Nu s-a putut stoca fisierul " + fileName, ex);
33     }
34 }
35 }
```

Figura 5.3. Logica pentru stocarea fizică a fișierelor și gestionarea identificatorilor unici în `FileStorageService`.

Metoda `storeFile` preia un obiect de tip `MultipartFile` trimis de către aplicația client. Pentru a preveni conflictele de denumire a fișierelor salvate pe disc, codul utilizează clasa `UUID` din pachetul nativ Java. Se generează astfel un identificator unic universal de 36 de caractere, la care se atașează extensia originală a imaginii. Fișierul este copiat ulterior în directorul de stocare configurat local pe server. Numele unic generat este returnat pentru a fi salvat în tabelul dosarului

de daună, asigurând o trasabilitate perfectă între înregistrarea din MySQL și fișierul fizic de pe disc.

5.2.3. Implementarea serviciilor REST

Expunerea funcționalităților către exterior se realizează prin intermediul serviciilor web de tip REST. Aceste componente au rolul de a intercepta cererile asincrone lansate din interfața grafică și de a returna răspunsurile într-un format standardizat. Interacțiunea se bazează exclusiv pe transferul de date în format JSON, un standard ușor și independent de platformă. Serviciile sunt mapate în pachetul controller. Ele folosesc adnotările specifice framework-ului Spring Boot pentru a gestiona rutele de acces. Acest nivel nu conține logică de business. Rolul său principal este de a valida intrările și de a redirecționează fluxurile informaționale către straturile inferioare ale aplicației.

Pentru a exemplifica arhitectura unui astfel de punct de acces programabil, se prezintă o metodă reprezentativă din clasa `DamageCaseController`:

```
12
13 @RestController @ Sorina
14 @RequestMapping("/api/damages")
15 @CrossOrigin(origins = "http://localhost:5173")
16 public class DamageController {
17
18     private final DamageCaseRepository caseRepository; 3 usages
19     private final DamagePartRepository partRepository; 2 usages
20     private final MessageRepository messageRepository; 2 usages
21     private final RestTemplate restTemplate; 2 usages
22
23     public DamageController(DamageCaseRepository caseRepository, @ Sorina
24                             DamagePartRepository partRepository,
25                             MessageRepository messageRepository) {
26         this.caseRepository = caseRepository;
27         this.partRepository = partRepository;
28         this.messageRepository = messageRepository;
29         this.restTemplate = new RestTemplate();
30     }
31
32     @PostMapping("/analyze") @ Sorina
33     public ResponseEntity<AiResponseDTO> analyze(@RequestBody Map<String, String> request) {
34         String userText = request.get("text");
35         String pythonUrl = "http://localhost:8000/analyze";
36
37         AiResponseDTO ai = restTemplate.postForObject(pythonUrl, request, AiResponseDTO.class);
38     }
```

Figura 5.4. Expunerea endpoint-ului REST și logica de interconectare cu serviciul extern Python AI în clasa `DamageController`.

Analiza fragmentului de cod evidențiază utilizarea adnotării `@RestController`, care indică faptul că fiecare metodă va returna direct datele serializate în corpul răspunsului HTTP. Ruta de baza este definită prin `@RequestMapping("/api/cases")`. Un element critic pentru funcționarea întregului sistem decuplat îl reprezintă adnotarea `@CrossOrigin`. Aceasta permite aplicației React, care rulează local pe portul 5173, să comunice cu backend-ul fără a fi blocată de politicile de securitate native ale browserului. Metoda `createCase` utilizează `@PostMapping` pentru a intercepta acțiunile de adăugare a unui dosar nou. Datele sosite prin rețea sunt mapate automat pe obiectul

Java cu ajutorul `@RequestBody`, în timp ce `@Valid` garantează respectarea constrângerilor de integritate înainte de execuția propriu-zisă a codului. Răspunsul este împachetat într-un obiect `ResponseEntity`, transmițând către frontend starea controlată 201 Created.

5.3. Implementarea bazei de date MySQL

Implementarea bazei de date a constat în rularea scripturilor DDL pe serverul relațional MySQL pentru a crea structura fizică a tabelelor. Fiecare coloană a fost configurată cu tipul de date optim pentru a reduce amprenta pe disc și pentru a asigura o procesare rapidă a interogărilor complexe. Constrângerile de cheie străină au fost definite explicit pentru a împiedica alterarea datelor.

Pentru a ilustra modul în care au fost transpuse constrângerile relaționale direct în sintaxa SQL, se prezintă scriptul de creare a tabelului central `damage_cases`:

```
12
13 @Entity @Sorina
14 @Table(name = "damage_cases")
15 @Getter
16 @Setter
17 @NoArgsConstructor
18 @AllArgsConstructor
19 public class DamageCase {
20
21     @Id
22     @GeneratedValue(strategy = GenerationType.IDENTITY)
23     private Long id;
24
25     private LocalDateTime creationDate;
26
27     private String status;
28
29     @ManyToOne(fetch = FetchType.LAZY)
30     @JoinColumn(name = "vehicle_id", nullable = false)
31     private Vehicle vehicle;
32
33     @ManyToOne(fetch = FetchType.LAZY)
34     @JoinColumn(name = "user_id", nullable = false)
35     private User creator;
36
37     @OneToMany(mappedBy = "damageCase", cascade = CascadeType.ALL, orphanRemoval = true)
38     private List<DamagePart> damagedParts = new ArrayList<>();
39
40     @PrePersist @Sorina
41     protected void onCreate() {
42         creationDate = LocalDateTime.now();
43     }
44 }
```

Figura 5.5. Definirea entității `DamageCase` și maparea structurală a relațiilor de tip Many-to-One și One-to-Many.

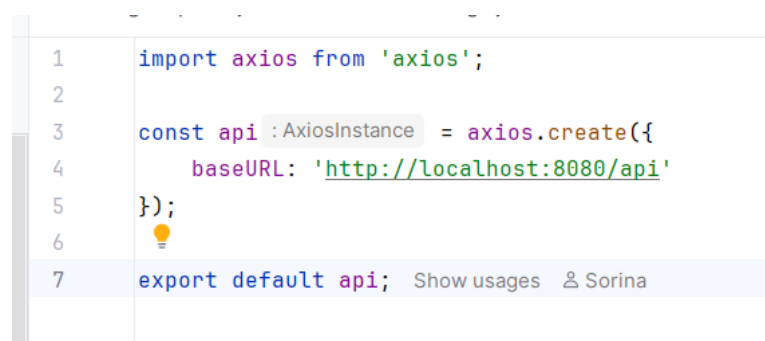
Tabelul utilizează o cheie primară auto-incrementată de tip `BIGINT` pentru a asigura suportul pentru un volum mare de înregistrări. Coloanele `user_id` și `vehicle_id` funcționează ca chei străine. Constrângerile au fost denumite explicit prin clauza `CONSTRAINT` și folosesc directiva `ON DELETE RESTRICT`. Această opțiune garantează că niciun utilizator sau vehicul nu poate fi șters din baza de date atâta timp cât există un dosar de daună asociat, eliminând complet riscul apariției înregistrărilor orfane.

5.4. Implementarea interfeței utilizator folosind React.js

Interfața grafică a fost construită ca o aplicație de tip Single Page Application (SPA), cu o structură modulară alcătuită din componente funcționale izolate ce își gestionează propria stare internă. Comunicarea cu backend-ul Spring Boot se realizează asincron, prin intermediul bibliotecii Axios.

5.4.1. Modulul de autentificare și autorizare

Managementul conexiunilor și securitatea la nivelul interfeței grafice se bazează pe centralizarea comunicării cu serverul de backend. Pentru a asigura un flux de date curat și ușor de întreținut, s-a optat pentru crearea unei instanțe unice și dedicate a bibliotecii Axios. Această configurare stabilește un punct de control comun pentru toate cererile HTTP lansate din paginile aplicației React.js, structura de cod fiind redată în Figura 5.6.



```
1 import axios from 'axios';
2
3 const api :AxiosInstance = axios.create({
4   baseURL: 'http://localhost:8080/api'
5 });
6
7 export default api; Show usages Sorina
```

Figura 5.6. Configurarea instanței centralizate Axios.

Instanța modulară definește proprietatea baseURL, direcționând automat toate apelurile asincrone către portul 8080 pe care rulează serverul Spring Boot. Soluția elimină definitiv necesitatea scrierii adresei complete a serverului în fiecare componentă de interfață în parte. Acest mod de lucru izolează configurarea de rețea. Dacă adresa serverului se schimbă în faza de producție, modificarea se va efectua într-un singur loc, asigurând o mentenanță optimă a întregului sistem software. Preluarea datelor de autentificare din formulare se face prin intermediul acestui obiect exportat, securizând indirect comunicarea.

5.4.2. Modulul de gestionare a clienților

Utilizatorii cu rol administrativ au acces la un panou centralizat unde datele sunt preluate de la endpoint-ul /api/users și afișate într-un tabel interactiv oferit de Material UI. Modulul include funcții de căutare și filtrare locală: operatorul poate introduce numele unui client, iar interfața React actualizează instantaneu rândurile tabelului fără a reîncărca pagina. Această filtrare dinamică se bazează pe prelucrarea matricei de stare, proces ilustrat în Figura 5.7.

```

98
99     const filteredUsers : any[] = users.filter(user =>
100         user.username?.toLowerCase().includes(searchTerm.toLowerCase()) ||
101         user.email?.toLowerCase().includes(searchTerm.toLowerCase())
102     );
103

```

Figura 5.7. Filtrarea locală a listei de clienți în React.

Logica utilizează metoda nativă de filtrare a limbajului JavaScript pentru a compara șirurile de caractere introduse de operator cu numele salvate în starea aplicației. Orice modificare a termenului de căutare declanșează instantaneu o nouă randare a tabelului grafic. Interfața utilizator oferă o experiență de utilizare fluentă și fără întârzieri perceptibile.

5.4.3. Modulul de gestionare a vehiculelor

Vehiculele înregistrate sunt afișate sub formă de carduri grafice, fiecare conținând sigla mărcii, modelul și numărul de înmatriculare. Formularul de adăugare include validare în timp real direct în browser. Dacă seria VIN introdusă nu are exact 17 caractere, butonul de trimitere este dezactivat automat și se afișează un mesaj de eroare sub câmpul respectiv. Structura de verificare a lungimii este redată în Figura 5.8.

```

let vin;
const isVinInvalid : boolean = vin.length !== 17;
const errorText : string = isVinInvalid ? "Seria VIN trebuie sa aiba exact 17 caractere" : "";

```

Figura 5.8. Validarea seriei VIN în formularul React.

Variabila booleană controlează în mod direct starea de activare a elementelor vizuale din formular. Nicio cerere greșită nu poate părăsi interfața grafică pentru a pleca spre serverul de backend. Utilizatorul primește o reacție vizuală imediată.

5.4.4. Modulul de administrare a dosarelor de daună

Procesul de deschidere și înregistrare a unui dosar nou beneficiază de o automatizare inteligentă prin integrarea sistemului ALPR (Automatic License Plate Recognition). Utilizatorul încarcă o fotografie a autovehiculului avariat, iar aplicația frontend transmite fișierul binar către serverul de backend pentru extragerea computerizată a numărului de înmatriculare. Această operațiune asincronă utilizează un obiect special de tip FormData pentru a încapsula imaginea și a o trimite securizat prin protocolul HTTP către endpoint-ul /api/client/process-plate. Implementarea tehnică a procesului de încărcare și prelucrare a răspunsului este detaliată în Figura 5.9.

```

16   const handlePlateUpload = async (event) : Promise<void> => { Show usages  Sorina
17     const file = event.target.files[0];
18     if (!file) return;
19
20     const formData : FormData = new FormData();
21     formData.append( name: 'file', file);
22
23     setLoading( value: true);
24     try {
25       const response : AxiosResponse<any> = await axios.post( url: 'http://localhost:8080/api/client/process-plate', formData, config: {
26         headers: { 'Content-Type': 'multipart/form-data' }
27       });
28
29       const data = typeof response.data === 'string' ? JSON.parse(response.data) : response.data;
30       if (data.status === 'success') {
31         setPlateNumber(data.plate);
32       } else {
33         alert('Nu s-a putut detecta numărul. Introduceți manual.');

```

Figura 5.9. Transmiterea fotografiei și procesarea asincronă ALPR cu Axios.

Logica izolează primul fișier selectat din evenimentul de încărcare și blochează interfața prin activarea unei stări globale de încărcare. Deoarece răspunsul primit de la scriptul de analiză computațională poate sosi sub formă de șir de caractere primitiv sau obiect gata mapat, funcția evaluează dinamic tipul de date și efectuează conversia structurală prin metoda `JSON.parse`. Dacă proprietatea de status confirmă succesul recunoașterii optice, numărul extras este salvat în starea locală prin funcția `setPlateNumber`, completând automat câmpul din formular. Tratarea riguroasă a erorilor interceptează defecțiunile de rețea și asigură eliberarea resurselor în blocul final, garantând o experiență fluidă și predictibilă.

5.4.5. Modulul de programare și monitorizare a reparațiilor

Acest ecran oferă o privire de ansamblu detaliată asupra evoluției lucrărilor tehnice și mecanice asociate unui anumit dosar de daună. Funcționalitatea se bazează pe o componentă vizuală interactivă ce reflectă stadiul curent al vehiculului, asigurând o sincronizare constantă cu baza de date prin interogări periodice efectuate la nivelul frontend-ului. Pentru a gestiona schimbările de stare și a asigura o reprezentare cromatică intuitivă pentru fiecare etapă operațională a dosarului, s-a implementat o funcție de mapare dinamică denumită `getStatusColor`, structura acestuia fiind redată în Figura 5.10.

```

15   const getStatusColor = (status) : string => { Show usages  Sorina
16     switch (status?.toUpperCase()) {
17       case 'APROBAT': return 'success';
18       case 'FINALIZAT': return 'success';
19       case 'IN_ANALIZA': return 'info';
20       case 'IN_REPARATIE': return 'secondary';
21       case 'IN_ASTEPTARE': return 'warning';
22       case 'RESPINS': return 'error';
23       default: return 'default';
24     }
25   };

```

Figura 5.10. Logica de mapare cromatică a statusurilor în React.

Prin intermediul acestei structuri de tip selecție (switch), valoarea textuală a statusului preluat din baza de date este convertită automat într-o etichetă de culoare nativă din paleta Material UI. Această abordare garantează o flexibilitate ridicată, asociind nuanțe specifice (de exemplu, success pentru dosarele aprobate sau finalizate, respectiv error pentru cele respinse) direct proprietăților componentelor de interfață.

Actualizarea datelor și sincronizarea ecranului cu realitatea fizică din teren sunt realizate prin funcția asincronă `fetchReparationStatus`, apelată în fundal la un interval fix de zece secunde prin intermediul unui hook `useEffect`. Rutina execută o cerere HTTP asincronă către serverul de backend, mecanismul fiind detaliat în Figura 5.11.

```
25     const fetchReparationStatus = (id) :void => { Show usages new *
26         api.get( url: `/client/reports/${id}/status`) Promise<AxiosResponse<...>>
27         .then(res : AxiosResponse<any> => {
28             setStatus(res.data.status);
29             setError( value: null);
30         }) Promise<void>
31         .catch(err => {
32             console.error(err);
33             setError( value: 'Nu s-a putut prelua statusul actualizat.');
```

Figura 5.11. Rutina de preluare asincronă a stării dosarului.

Logica utilizează promisiuni JavaScript pentru a procesa răspunsul primit de la endpoint-ul `/client/reports/${id}/status`. La recepționarea cu succes a pachetului JSON, funcția actualizează starea locală a aplicației prin `setStatus`, determinând re-arendarea instantă a indicatorilor grafici de pe ecran. Blocul `catch` garantează izolarea erorilor de comunicație, prevenind întreruperea execuției codului prin captarea excepțiilor și setarea unui mesaj de avertizare controlat, în timp ce instrucțiunea din blocul `finally` asigură oprirea indicatorului de încărcare circular prin trecerea stării `setLoading` pe valoarea fals.

5.4.6. Modulul de rapoarte și statistici

Modulul afișează diagrame circulare și histogramme cu volumul total al daunelor, generate cu ajutorul unor biblioteci grafice dedicate. Funcționalitatea centrală rămâne însă generarea și descărcarea automată a raportului global sau a notei de constatare sub formă de document oficial. La acționarea butonului de export, aplicația apelează o funcție specializată care utilizează metoda nativă `window.open` pentru a naviga direct către endpoint-ul de export furnizat de serverul de backend. Acest mecanism asigură deschiderea unei noi file și declanșarea imediată a transferului de date, procesul fiind redat în Figura 5.12.

```

27     const handleDownloadPDF = () :void => { Show usages  Sorina
28         // Deschide direct link-ul de download din backend
29         window.open( url: 'http://localhost:8080/api/admin/reports/export/pdf', target: '_blank');
30     };

```

Figura 5.12. Declanșarea descărcării fișierului PDF prin window.open.

Invocarea metodei direct pe URL-ul expus de aplicația Spring Boot permite transmiterea instantanee a fișierului PDF, fără a mai fi necesară prelucrarea fluxului binar în memoria locală a aplicației React. Browserul interceptează automat antetul de tip attachment trimis de server și descarcă documentul binar direct pe calculatorul utilizatorului, oferind o fișă curată și gata de tipărire.

5.5. Integrarea componentelor aplicației

Faza de integrare a vizat conectarea interfeței React cu serverul Spring Boot și baza de date MySQL. Fluxul urmează modelul clasic client-server: interfața preia acțiunile utilizatorului, lansează cereri REST, serverul le procesează în straturile sale interne, interoghează sau modifică tabelele din MySQL și returnează răspunsul în format JSON. Întreg acest circuit se desfășoară asincron, menținând coerența datelor pe toate nivelurile fără a bloca interacțiunea cu utilizatorul. Un exemplu clar de integrare la nivelul configurării securității pe server este prezentat în Figura 5.13.

```

1 package ro.university.vehicledamagemanager.config;
2
3 > import ...
4
5
6
7
8 @Configuration  Sorina
9 public class WebConfig {
10
11     @Bean  Sorina
12     public WebMvcConfigurer corsConfigurer() {
13         return addCorsMappings(registry) -> {
14             registry.addMapping( pathPattern: "/*")
15                 .allowedOriginPatterns(["http://localhost:5173", "http://localhost:3000"])
16                 .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
17                 .allowedHeaders("*")
18                 .allowCredentials(true);
19         };
20     };
21 }
22
23
24

```

Figura 5.13. Configurarea politicii CORS în backend-ul Spring Boot.

Fără această configurare explicită făcută pe server, sistemul de securitate al browserului ar bloca orice încercare de schimb de date din cauza politicilor de origine diferite. Această clasă permite legarea straturilor într-un tot unitar și funcțional.

Capitolul 6. Utilizarea sistemului informatic

6.1. Autentificarea în aplicație

Accesul la resursele platformei este condiționat de parcurgerea procesului de verificare a identității. Utilizatorul introduce adresa de email și parola aferente contului în formularul de autentificare prezentat în Figura 6.1.

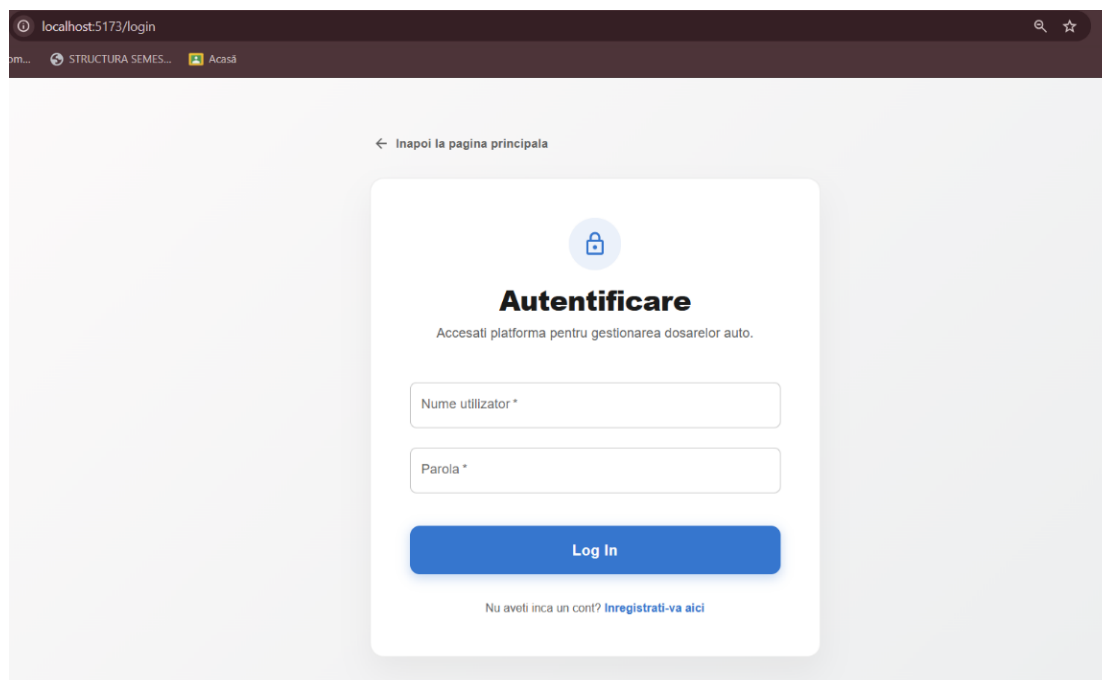


Figura 6.1. Ecranul de autentificare.

După apăsarea butonului de conectare, datele sunt transmise securizat către backend prin ruta dedicată. Dacă credențialele sunt corecte, sistemul generează un token JWT și redirecționează utilizatorul către tabloul de bord principal. În caz contrar, aplicația afișează un mesaj de avertizare și solicită reintroducerea datelor, rutele interne rămânând protejate împotriva accesului neautorizat.

6.2. Gestionarea datelor despre clienți și vehicule

Odată autentificat, utilizatorul cu drepturi de administrator are acces la panourile centrale de control și supervizare a sistemului. Primul modul esențial este reprezentat de interfața Management Utilizatori, ilustrată în Figura 6.2. Acest ecran pune la dispoziție o listă tabelară complexă ce cuprinde datele de identificare ale tuturor conturilor înregistrate în baza de date, structurată pe coloane specifice: ID-ul unic alocat precedent, numele de utilizator, adresa de email și rolul operațional curent.

Modificarea drepturilor de acces se realizează direct din tabel prin intermediul unui meniu derulant de tip Select, ce permite comutarea rapidă între rolurile de CLIENT, OPERATOR, INSPECTOR, SERVICE sau ADMIN. Coloana dedicată acțiunilor utilizează elemente de tip

Generează/IconButton pentru salvarea modificărilor sau ștergerea contului, iar în cazul utilizatorilor cu rolul de CLIENT, sistemul randează dinamic pictograme suplimentare pentru accesarea rapidă a vehiculelor înregistrate și descărcarea fișei de daunalitate PDF. Filtrarea rapidă a datelor este asigurată de un câmp de căutare textual amplasat superior, iar adăugarea de noi operatori se realizează prin acționarea butonului verde Utilizator Nou.

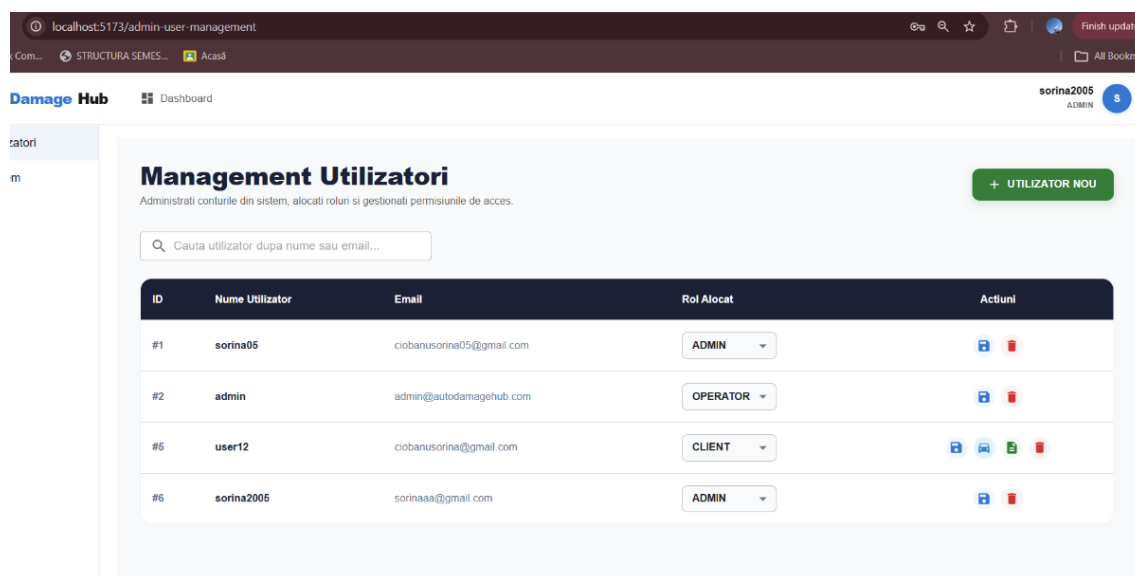


Figura 6.2. Interfața de management și control a utilizatorilor în sistem.

Cel de-al doilea panou administrativ major, corelat direct cu activitatea operațională a platformei, este reprezentat de modulul Management Global Dosare Daune, detaliat în Figura 6.3. Acest ecran centralizează toate solicitările transmise de către clienți, oferind personalului autorizat o privire de ansamblu exhaustivă asupra cazurilor active și arhivate.

Tabelul interactiv randează informații tehnice cruciale, organizate în coloane precum ID-ul dosarului, descrierea detaliată a incidentului rutier, numărul de înmatriculare identificat computațional sau introdus manual, numele și email-ul proprietarului de dosar. Pentru o identificare vizuală imediată și o organizare eficientă, statusul curent al fiecărui caz este randat sub forma unor componente grafice de tip Chip, colorate diferențiat pe baza stării stocate în baza de date (portocaliu pentru IN_AȘTEPTARE, albastru pentru IN_ANALIZĂ și verde pentru APROBAT). În partea din dreapta sus a ecranului este poziționat butonul albastru Export Raport PDF, care declanșează extragerea instantanee a tuturor înregistrărilor din tabelele MySQL și generarea documentului oficial binar de audit global.

Management Global Dosare Daune						EXPORT RAPORT PDF
ID Dosar	Detalii/Descriere Incident	Numar Inmatriculare	Proprietar Dosar (Client)	Email Client	Status Curent	
1	Lovitura in partea frontala, bara de protectie desprinsa si far stanga spart in parcare.	B123ABC	admin	admin@autodamagehub.com	IN_ASTEPTARE	
2	Zgarietura adanca pe toata lungimea portierei dreapta spate, provocata de vandalism.	CJ99XYZ	admin	admin@autodamagehub.com	IN_ANALIZA	
3	Parbriz fisurat in forma de panza de paianjen din cauza unei pietre sarite pe autostrada.	TM44BRD	admin	admin@autodamagehub.com	APROBAT	
4	Lovitura in partea frontala, bara de protectie desprinsa si far stanga spart in parcare.	B123ABC	user12	ciobanusorina@gmail.com	IN_ASTEPTARE	
5	Zgarietura adanca pe toata lungimea portierei dreapta spate, provocata de vandalism.	CJ99XYZ	user12	ciobanusorina@gmail.com	IN_ANALIZA	
6	Parbriz fisurat in forma de panza de paianjen din cauza unei pietre sarite pe autostrada.	TM44BRD	user12	ciobanusorina@gmail.com	APROBAT	
7	Lovitura in partea frontala, bara de protectie desprinsa si far stanga spart in parcare.	B123ABC	user12	ciobanusorina@gmail.com	IN_ASTEPTARE	

Figura 6.3. Modulul de management global al dosarelor de daună.

6.3. Administrarea dosarelor de daună

Procesul operațional de raportare a unui incident și înregistrare a unui caz nou este structurat sub forma unui asistent ghidat (Stepper orizontal) integrat în componenta de frontend, menit să simplifice fluxul de interacțiune pentru utilizatorii cu rol de CLIENT. Interfața grafică a acestui modul este ilustrată detaliat în Figura 6.4. Asistentul împarte întregul proces în trei etape distincte și succesive: Identificare Vehicul, Încărcare Fotografii Daune și Detalii Suplimentare (AI Chat), oferind utilizatorului o trasabilitate clară și predictibilă a acțiunilor sale.

Figura 6.4. Interfața modulului Raportare daună nouă.

În cadrul primului pas, care vizează identificarea vehiculului, sistemul utilizează modulul de recunoaștere optică computațională ALPR. Utilizatorul are posibilitatea de a acționa butonul central albastru „Încarcă poza număr”, acțiune ce declanșează încărcarea asincronă a unei imagini cu plăcuța de înmatriculare, fișierul binar fiind procesat instantaneu de algoritmul din backend pentru a extrage caracterele și a completa automat datele mașinii.

Pentru a asigura o toleranță ridicată la defecțiuni și o flexibilitate sporită a platformei, sub zona de încărcare este poziționat un câmp textual secundar („Sau introdu manual numărul”), permițând asiguratului să corecteze sau să introducă manual numărul de înmatriculare în situația în care condițiile de luminozitate sau unghiul fotografiei împiedică detecția automată a componentelor de către inteligența artificială. Navigarea între etape este controlată prin butoanele operaționale „Înapoi” și „Continuă”, plasate în colțul din dreapta-jos al panoului principal, interfața blocând sau activând aceste elemente în mod dinamic în funcție de starea de validare a datelor introduse.

6.4. Monitorizarea stadiului lucrărilor

Urmărirea evoluției lucrărilor tehnice și a stadiului de procesare a dosarului se realizează prin intermediul modului Monitorizare Progres Reparație, disponibil la ruta /track-repair. Interfața grafică este proiectată special pentru a oferi o transparență totală asiguraților și personalului tehnic, reducând fluxul de solicitări telefonice către recepția service-ului. Pentru o prezentare fidelă a funcționalităților, modulul integrează în partea dreapta-sus un selector dropdown („Selectează Dosar Client”) și un panou de simulare a schimbărilor de stare, aspecte ilustrate în Figura 6.5 și Figura 6.6.

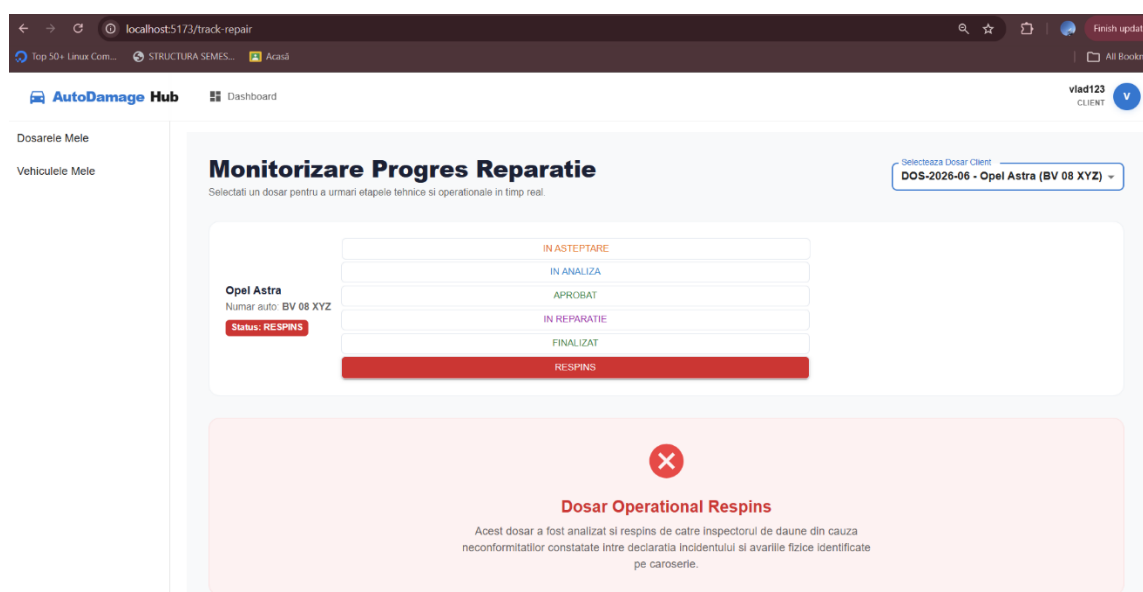


Figura 6.5. Interfața de monitorizare în cazul unui dosar respins operațional.

În situația în care un caz este invalidat de către personalul administrativ, cum este exemplul dosarului DOS-2026-06 pentru vehiculul Opel Astra (număr auto BV 08 XYZ) prezentat în Figura 6.5, interfața reacționează dinamic. Modificarea statusului în RESPINS determină ascunderea axei standard a timpului și randarea unui panou dedicat de avertizare, centrat și marcat vizual cu o pictogramă de tip CancelIcon de culoare roșie. Mesajul detaliat afișat în partea inferioară informează utilizatorul cu privire la neconformitățile constatate între declarația incidentului și avariile fizice identificate pe caroserie, oferind o justificare clară a deciziei luate de inspector.

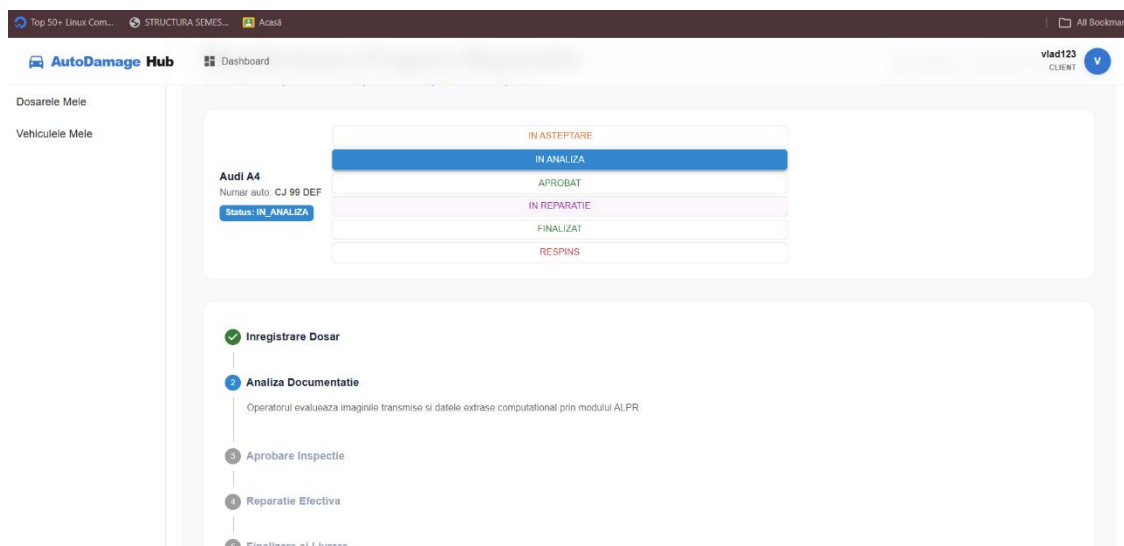


Figura 6.6. Componenta Stepper vertical și evoluția etapelor pentru un dosar activ.

Pentru dosarele aflate în curs de soluționare, cum este cazul vehiculului Audi A4 (număr auto CJ 99 DEF) ilustrat în Figura 6.6, sistemul utilizează o componentă de tip Stepper vertical din suita Material UI. Fiecare etapă operațională (Înregistrare Dosar, Analiză Documentație, Aprobare Inspectie, Reparație Efectivă, Finalizare și Livrare) reprezintă un nod distinct pe această axă. Etapele parcurse cu succes primesc automat un indicator circular verde de confirmare, în timp ce etapa curentă devine activă, schimbându-și culoarea în albastru pe baza logicii `getStatusColor` și extinzând o zonă textuală ce descrie procesul curent de evaluare a imaginilor ALPR de către operator.

6.5. Generarea rapoartelor

Funcționalitatea centrală a acestui modul vizează generarea și descărcarea documentelor oficiale de audit tehnic, aplicația oferind suport atât pentru rapoarte globale, cât și pentru fișe individuale per client. Interfața de declanșare a exportului pentru un anumit asigurat este integrată direct în panoul de management al utilizatorilor, acțiunea fiind evidențiată în Figura 6.7. Operatorul poate identifica vizual elementele de interacțiune dedicate conturilor de tip CLIENT, unde sistemul randează o pictogramă verde însoțită de un indicator contextual text de tip Tooltip cu mesajul „Descarcă Fișă Daune PDF”.

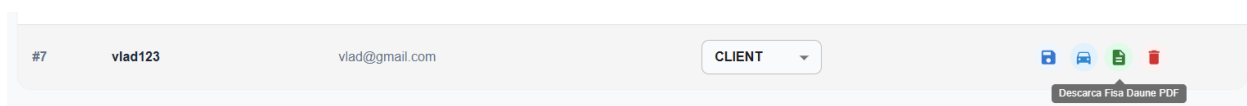


Figura 6.7. Activarea butonului de export pentru fișa de daunalitate a clientului.

La acționarea acestui element grafic, aplicația din frontend inițiază o cerere web securizată către serverul Spring Boot. Acesta interoghează baza de date relațională și assemblează informațiile specifice contului într-un document structurat prin intermediul bibliotecii OpenPDF. Rezultatul final al acestui flux operațional este ilustrat în Figura 6.8, unde este prezentat documentul binar

descărcat în mod automat de către browser și stocat local sub denumirea dinamică fisa_daune_client_7.pdf”.

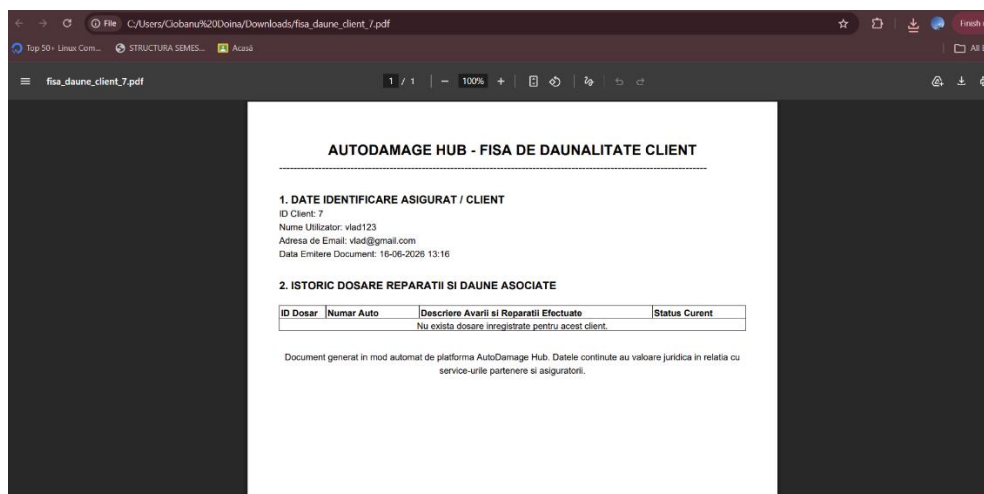


Figura 6.8. Documentul PDF generat reprezentând fișa de daunalitate a clientului în browser.

Fișierul generat respectă un format enterprise riguros. Prima secțiune, intitulată „1. DATE IDENTIFICARE ASIGURAT / CLIENT”, centralizează atributele de bază ale utilizatorului, precum ID-ul unic din sistem, numele de utilizator, adresa de email și amprenta temporară exactă a emiterii documentului de constatare. Cea de-a doua secțiune, „2. ISTORIC DOSARE REPARAȚII ȘI DAUNE ASOCIATE”, încorporează un tabel tehnic structurat pe coloane destinate ID-ului de dosar, numărului auto, descrierii detaliate a avariilor și statusului curent. În situația în care utilizatorul selectat nu are încă dosare de daună înregistrate în sistem, aplicația frontend și serverul tratează această excepție prin afișarea unui mesaj de informare curat și aliniat: „Nu există dosare înregistrate pentru acest client.”, asigurând astfel validitatea și acuratețea rapoartelor oficiale emise.

6.6. Exemple de scenarii de utilizare

Pentru a ilustra funcționarea de ansamblu a sistemului, au fost simulate două scenarii complete care acoperă principalele fluxuri operaționale.

Scenariul 1: Deschiderea și evaluarea unui dosar de daună

- Actori: Clientul și Inspectorul de daune.
- Pasul 1: Clientul se autentifică în aplicație cu adresa de email și parola aferente contului.
- Pasul 2: Accesează modulul de vehicule și selectează mașina implicată în accident.
- Pasul 3: Completează formularul de daună, descrie incidentul și încarcă fotografiile cu bara de protecție avariata, apoi trimite dosarul.
- Pasul 4: Inspectorul se autentifică cu rol administrativ și accesează lista globală de dosare.

- Pasul 5: Deschide dosarul creat, examinează imaginile, apoi aprobă cazul pentru reparație.

Scenariul 2: Actualizarea reparației și generarea notei de constatare

- Actori: Operatorul de service, Clientul și Managerul.
- Pasul 1: Operatorul caută mașina după numărul de înmatriculare și deschide panoul de monitorizare.
- Pasul 2: Modifică statusul în „În lucru la vopsitorie”. Clientul observă imediat actualizarea nodului pe axa timpului.
- Pasul 3: La finalizarea lucrărilor, operatorul marchează stadiul ca „Reparație finalizată”.
- Pasul 4: Managerul accesează modulul de rapoarte și selectează dosarul finalizat.
- Pasul 5: Acționează butonul de export, iar sistemul generează și descarcă automat nota de constatare în format PDF.

Capitolul 7. Evaluarea și analiza rezultatelor și beneficiilor aplicației

7.1. Beneficiile utilizării sistemului informatic

Introducerea unei platforme digitale destinate gestiunii avariilor auto determină transformări profunde în privința administrării dosarelor, eliminând procedurile clasice marcate de birocrație excesivă și activități fragmentate. Întregul circuit al informațiilor este reunit de aplicație într-un spațiu centralizat și sigur, fapt care elimină împrăștierea documentelor prin salvarea ordonată a actelor doveditoare, a materialelor foto-video și a devizelor de lucru în cadrul aceleiași baze de date. Coerența datelor vizualizate de utilizatori este susținută prin structurarea corectă a schemelor relaționale, aspect ce previne răspândirea unor detalii eronate sau parțiale.

Fiecare grup de utilizatori care interacționează cu sistemul resimte direct avantajele oferite de structura organizată pe trei niveluri. Interfața realizată cu ajutorul React.js le asigură clienților o vizibilitate excelentă asupra operațiunilor, simplificând demersurile administrative prin interacțiuni ce se desfășoară în fundal. Constituie un câștig real pentru experiența utilizatorului opțiunea de a iniția online un dosar de daună și de a monitoriza parcursul reparațiilor în timp real, fiind eliminate deplasările la birourile asigurătorilor sau apelurile telefonice insistente. Pe lângă beneficiile aduse publicului, programul le oferă operatorilor și inspectorilor o scurtare substanțială a timpilor de lucru datorită atribuirii automate a dosarelor deschise. Informațiile transmise de clienți devin accesibile instantaneu în panoul inspectorului desemnat, nemaifiind necesare copierile manuale, element ce îndepărtează riscul greșelilor de tastare și oferă o trasabilitate clară întregului proces de verificare.

Atelierele de reparații aflate în parteneriat pot urmări, de asemenea, hotărârile tehnice adoptate de inspectori, acestea fiind salvate pe loc în baza de date partajată. Trimiterea accelerată a devizelor estimate și a documentelor anexe direct către serverul central micșorează perioadele de așteptare din faza de plată, scăzând numărul de zile în care autoturismele rămân blocate în service. Corelarea straturilor din program garantează o funcționare stabilă a platformei, confirmând valoarea acestei soluții informatice în raport cu cerințele curente din domeniul asigurărilor.

7.2. Impactul asupra eficienței operaționale

Separarea clară dintre interfața grafică și nucleul logic al aplicației garantează că operațiile computaționale mai complexe, cum ar fi analiza devizelor sau interogările pe tabelele din baza de date MySQL, sunt gestionate exclusiv de serverul Spring Boot, fără a bloca interacțiunea utilizatorului în browser. Utilizarea serviciilor REST și a formatului JSON pentru schimbul de date reduce volumul traficului de rețea și permite componentelor din frontend să reacționeze rapid la modificările de stare, asigurând o experiență stabilă și funcțională inclusiv în condiții de încărcare mai mare.

Pentru a evidenția concret îmbunătățirile aduse la nivelul fluxului de lucru, tabelul următor compară etapele procesului clasic, bazat pe interacțiuni manuale, cu procesul digitalizat integrat în aplicația propusă:

Etapă operațională	Procesul tradițional (Manual)	Procesul digitalizat (Aplicația propusă)	Impact operațional
Deschiderea dosarului	Preluare telefonică, completare manuală de formulare tipărite pe hârtie, scanare ulterioară a documentelor.	Introducerea directă a datelor de către solicitant în interfața React, urmată de o verificare imediată la nivelul schemei.	Scăderea timpului necesar preluării cu un procent ce depășește 50 la sută.
Constatarea avariilor	Călătorii efectuate pe teren, redactarea pe hârtie a procesului verbal, descărcarea plus organizarea manuală a pozelor.	Trimiterea și afișarea imediată a imaginilor în sistem, urmată de completarea unui chestionar digital corelat cu entitățile Hibernate.	Îndepărtarea completă a drumurilor consumatoare de timp și trecerea la o stocare electronică permanentă.
Aprobarea devizului	Expedierea estimărilor prin poșta electronică, evaluări manuale repetate și primirea unui răspuns cu întârzieri mari.	Salvarea sumelor direct în secțiunea destinată atelierului, urmată de o avertizare vizuală și confirmare rapidă în ecranul inspectorului.	Scurtarea timpului dedicat acestei etape de la un interval de câteva zile la numai câteva ore.
Monitorizarea stadiului	Apeluri telefonice frecvente și discuții purtate permanent între păgubit, atelierul auto și societatea de asigurări.	Ecran de control dinamic ce primește date în timp real prin căutări optimizate bazate pe indici în MySQL.	Înlăturarea completă a blocajelor de comunicare și obținerea unei vizibilități totale asupra procesului.

Tabelul 7.1. Evaluarea comparativă a eficienței operaționale între procesul tradițional și soluția digitală propusă.

7.3. Limitările aplicației dezvoltate

O evaluare corectă a platformei informatice create cere evidențierea punctelor sale slabe, elemente care nu blochează funcțiile de bază cerute inițial, dar pot aduce dificultăți într-un spațiu real de lucru cu foarte mulți utilizatori activi în același timp. O primă problemă tehnică se referă la modul în care sunt administrate fișierele de mari dimensiuni. Pozele trimise de utilizatori se salvează pe hard diskul serverului ori sunt indicate prin legături de tip text în tabelele MySQL, metodă care își pierde eficiența când volumul de date crește rapid. În situația în care mulți utilizatori ar folosi aplicația concomitent și ar trimite fișiere de mari dimensiuni, spațiul disponibil pe server s-ar putea epuiza, fapt ce ar încetini viteza ecranelor și ar îngreuna salvarea periodică a bazei de date.

Chiar dacă programul colectează datele și le prezintă clar utilizatorilor, confirmarea finală a devizelor depinde integral de hotărârea inspectorului, aspect ce menține o dependență strânsă de factorul uman, detaliu considerat un dezavantaj al versiunii curente. Platforma nu include un instrument de scanare automată a imaginilor sau o regulă matematică prin care să se verifice dacă tarifele adăugate de service respectă sumele recomandate de fabrică. Lipsa unor verificări automate în faza de evaluare tehnică expune procesul unor aprecieri personale și poate aduce diferențe mari de costuri între cazuri aproape identice.

Software-ul rulează în acest moment ca o structură izolată, neavând legături directe cu alte programe utilizate în acest domeniu economic. Interfața REST realizată în Spring Boot nu comunică cu bazele de date ale societăților de asigurare ori cu registrele naționale ale poliției, situație ce împiedică controlul automat al valabilității polițelor auto obligatorii sau facultative. Angajatul este obligat să caute manual aceste informații pe site-uri externe, acțiune care extinde timpul de lucru și arată că este nevoie de corelări tehnice ulterioare pentru ca aplicația să acopere toate cerințele pieței.

7.4. Direcții de dezvoltare ulterioară

Având ca punct de plecare problemele observate, pot fi stabilite căi clare de evoluție capabile să schimbe prototipul actual într-un produs robust, pregătit pentru utilizarea pe scară largă. O primă îmbunătățire utilă ar fi mutarea spațiului de stocare al fișierelor într-un sistem extern de tip Object Storage în cloud. Conectarea backend-ului Spring Boot cu servicii ca AWS S3 sau Azure Blob Storage ar permite ca imaginile și documentele trimise de oameni să fie salvate separat de baza de date MySQL, aceasta din urmă păstrând doar informațiile de legătură și detaliile de identificare. Această modificare ar micșora semnificativ timpul alocat copierii de siguranță, ar spori viteza serverului și ar ajuta sistemul să crească fără dificultăți odată cu înmulțirea cazurilor administrate.

Adăugarea unui instrument de analiză automată a fotografiilor prin metode de recunoaștere vizuală ar aduce un progres important programului. Prin folosirea unor algoritmi capabili să descopere zonele lovite ale caroseriei din pozele primite, aplicația ar putea oferi singură o estimare inițială cu piesele distruse și un calcul informativ al reparațiilor. Specialistul ar avea doar sarcina

de a analiza și de a valida o propunere deja realizată de calculator, nemaifiind nevoit să scrie totul manual, fapt ce ar diminua simțitor timpul pierdut cu inspecția tehnică.

Interconectarea platformei cu alte rețele informatice din industrie este o altă etapă utilă pentru eliminarea căutărilor manuale pe care le fac acum angajații. Corelarea API-ului din backend cu baze de date recunoscute, cum este Eurotax, ar aduce automat tarifele actualizate ale pieselor din fabrică, în timp ce o legătură cu registrele oficiale auto ar permite verificarea imediată a mașinilor și a asigurărilor valabile, eliminând accesarea separată a altor pagini web.

Dezvoltarea unei versiuni mobile native constituie o continuare firească pentru a sprijini utilizatorii care accesează platforma direct de pe teren. Cu ajutorul structurii oferite de React Native, secvențe mari din codul interfeței web actuale s-ar putea folosi din nou pentru a crea o aplicație adaptată sistemelor Android și iOS. Utilizatorii ar putea deschide un dosar chiar de la locul unui accident, realizând poze cu ajutorul camerei foto a telefonului și trimițând coordonatele geografice prin sistemul GPS, scurtând la maximum timpul scurs între producerea loviturii și înregistrarea ei oficială în sistem.

Concluzii

Finalizarea acestei aplicații destinate managementului integrat al daunelor auto reprezintă o dovadă clară a modului în care digitalizarea poate simplifica radical un domeniu afectat de birocrație. Acest proiect nu a rămas o simplă teorie academică de inginerie software, ci a devenit un instrument real și funcțional.

Alegerea unei arhitecturi decuplate, în care interfața React.js și backend-ul Spring Boot cooperează prin servicii web REST, s-a dovedit a fi cea mai bună decizie tehnică pentru stabilitatea sistemului. Această structură modulară demonstrează că platformele moderne pot oferi o viteză excelentă de răspuns fără să compromită securitatea sau izolarea datelor. Pe parcursul etapelor de dezvoltare, atenția s-a concentrat pe eliminarea erorilor umane ce apar inevitabil la completarea manuală a documentelor clasice. Prin stocarea centralizată și structurată a informațiilor în tabelele MySQL, s-a obținut o trasabilitate completă a fiecărui dosar operațional. Din acest moment, istoricul unui vehicul avariat poate fi consultat instantaneu, eliminând complet dosarele fizice rătăcite sau datele introduse duplicat.

Dincolo de avantajele tehnice evidente ale codului, aplicația reușește să schimbe fundamental experiența tuturor celor implicați în proces. Ea transformă interacțiunea, deseori tensionată și lentă, dintre clienți, inspecitori de daune și unitățile de service într-un parteneriat digital direct și previzibil. Clienții nu mai sunt nevoiți să efectueze zeci de apeluri telefonice pentru a afla stadiul reparației, deoarece panoul interactiv le oferă răspunsuri în timp real, crescând încrederea în serviciile oferite.

Privind în urmă la rezultatele obținute, consider că toate obiectivele tehnice și funcționale au fost atinse cu succes. Sistemul rulează fluid, iar automatizarea unor procese cheie, precum generarea rapidă a notelor de constatare în format PDF, confirmă maturitatea întregii platforme.

Desigur, aplicația reprezintă un fundament solid ce rămâne deschis către extinderi tehnologice de anvergură. O prioritate pentru versiunile următoare o constituie migrarea stocării fișierelor media către servere cloud specializate și integrarea unor module avansate de inteligență artificială și Computer Vision. Acest pas ar permite recunoașterea automată a severității daunelor direct din fotografiile încărcate de utilizatori, reducând și mai mult timpul alocat constatării tehnice. Conectarea prin API cu bazele de date naționale ale asigurătorilor și crearea unei aplicații mobile native vor întregi acest tablou, transformând prototipul actual într-un ecosistem digital de ultimă generație.

Bibliografie

- [1] Cline, M., & Kamalapurkar, K. (2021). Emerging Trends in Claims Transformation. Preluat pe 14 Ianuarie 2026, de pe Deloitte Insights: <https://www.deloitte.com/us/en/insights/industry/financial-services/insurance-claims-transformation.html>
- [2] Coalition Against Insurance Fraud. (2022). By the Numbers: Fraud Statistics. Preluat pe 22 Ianuarie 2026, de pe Insurance Fraud: <https://insurancefraud.org/fraud-stats/>
- [3] Solera Holdings. (2023). Audatex / Qapter Intelligent Estimating – Vehicle Claims Solutions. Preluat pe 3 Februarie 2026, de pe Solera: <https://www.claims.solera.com/products/intelligent-estimating/>
- [4] Guidewire Software. (2023). ClaimCenter – P&C Insurance Claims Management. Preluat pe 7 Februarie 2026, de pe Guidewire: <https://www.guidewire.com/products/claimcenter>
- [5] Snapsheet Inc. (2022). Virtual Claims Platform Overview. Preluat pe 11 Februarie 2026, de pe Snapsheet: <https://www.snapsheetclaims.com>
- [6] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Preluat pe 25 Februarie 2026, de pe University of California, Irvine: https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm
- [7] Oracle Corporation. (2023). Java SE 17 Documentation – The Java Virtual Machine. Preluat pe 18 Martie 2026, de pe Oracle: <https://docs.oracle.com/en/java/javase/17/>
- [8] VMware Inc. (2023). Spring Boot Reference Documentation (3.1.x). Preluat pe 18 Martie 2026, de pe Spring: <https://docs.spring.io/spring-boot/>
- [9] Meta Platforms, Inc. (2023). React – The library for web and native user interfaces. Preluat pe 5 Aprilie 2026, de pe React: <https://react.dev/>
- [10] Meta Platforms, Inc. (2023). Preserving and Resetting State – React Documentation. Preluat pe 12 Aprilie 2026, de pe React: <https://react.dev/learn/preserving-and-resetting-state>
- [11] Oracle Corporation. (2023). MySQL 8.0 Reference Manual – Chapter 15: The InnoDB Storage Engine. Preluat pe 21 Martie 2026, de pe MySQL: <https://dev.mysql.com/doc/refman/8.0/en/innodb-storage-engine.html>

[12] Microsoft Azure Architecture Center. (2023). N-tier architecture style. Preluat pe 2 Martie 2026, de pe Microsoft Learn: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier>